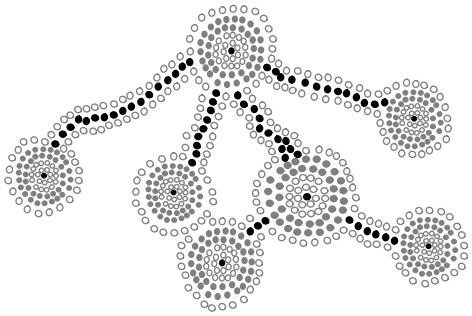
Chapter

11 Search Trees



Contents

11.1 Binary Search Trees 460
11.1.1 Navigating a Binary Search Tree 461
11.1.2 Searches 463
11.1.3 Insertions and Deletions 465
11.1.4 Python Implementation 468
11.1.5 Performance of a Binary Search Tree 473
11.2 Balanced Search Trees 475
11.2.1 Python Framework for Balancing Search Trees 478
11.3 AVL Trees 481
11.3.1 Update Operations 483
11.3.2 Python Implementation 488
11.4 Splay Trees 490
11.4.1 Splaying 490
11.4.2 When to Splay 494
11.4.3 Python Implementation 496
11.4.4 Amortized Analysis of Splaying ★ 497
11.5 (2,4) Trees 502
11.5.1 Multiway Search Trees 502
11.5.2 (2,4)-Tree Operations 505
11.6 Red-Black Trees 512
11.6.1 Red-Black Tree Operations 514
11.6.2 Python Implementation 525
11.7 Exercises 528

11.1 Binary Search Trees

In Chapter 8 we introduced the tree data structure and demonstrated a variety of applications. One important use is as a ***search tree*** (as described on page 332). In this chapter, we use a search tree structure to efficiently implement a ***sorted map***. The three most fundamental methods of a map M (see Section 10.1.1) are:

M[k]: Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError; implemented with __getitem__ method.

M[k] = v: Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k; implemented with __setitem__ method.

del M[k]: Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError; implemented with __delitem__ method.

The sorted map ADT includes additional functionality (see Section 10.3), guaranteeing that an iteration reports keys in sorted order, and supporting additional searches such as find_gt(k) and find_range(start, stop).

Binary trees are an excellent data structure for storing items of a map, assuming we have an order relation defined on the keys. In this context, a ***binary search tree*** is a binary tree T with each position p storing a key-value pair (k, v) such that:

- Keys stored in the left subtree of p are less than k.
- Keys stored in the right subtree of p are greater than k.

An example of such a binary search tree is given in Figure 11.1. As a matter of convenience, we will not diagram the values associated with keys in this chapter, since those values do not affect the placement of items within a search tree.

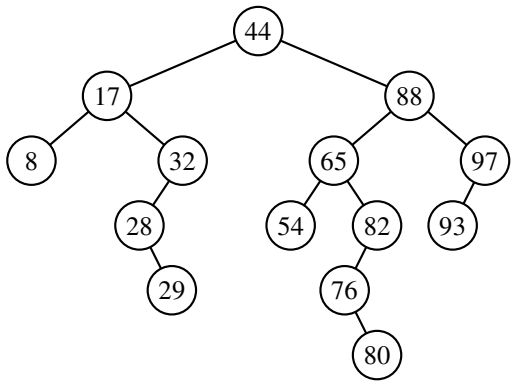


Figure 11.1: A binary search tree with integer keys. We omit the display of associated values in this chapter, since they are not relevant to the order of items within a search tree.

11.1.1 Navigating a Binary Search Tree

We begin by demonstrating that a binary search tree hierarchically represents the sorted order of its keys. In particular, the structural property regarding the placement of keys within a binary search tree assures the following important consequence regarding an ***inorder traversal*** (Section 8.4.3) of the tree.

Proposition 11.1: *An inorder traversal of a binary search tree visits positions in increasing order of their keys.*

Justification: We prove this by induction on the size of a subtree. If a subtree has at most one item, its keys are trivially visited in order. More generally, an inorder traversal of a (sub)tree consists of a recursive traversal of the (possibly empty) left subtree, followed by a visit of the root, and then a recursive traversal of the (possibly empty) right subtree. By induction, a recursive inorder traversal of the left subtree will produce an iteration of the keys in that subtree in increasing order. Furthermore, by the binary search tree property, all keys in the left subtree have keys strictly smaller than that of the root. Therefore, visiting the root just after that subtree extends the increasing order of keys. Finally, by the search tree property, all keys in the right subtree are strictly greater than the root, and by induction, an inorder traversal of that subtree will visit those keys in increasing order. ■

Since an inorder traversal can be executed in linear time, a consequence of this proposition is that we can produce a sorted iteration of the keys of a map in linear time, when represented as a binary search tree.

Although an inorder traversal is typically expressed using a top-down recursion, we can provide nonrecursive descriptions of operations that allow more fine-grained navigation among the positions of a binary search relative to the order of their keys. Our generic binary tree ADT from Chapter 8 is defined as a positional structure, allowing direct navigation using methods such as `parent(p)`, `left(p)`, and `right(p)`. With a binary *search* tree, we can provide additional navigation based on the natural order of the keys stored in the tree. In particular, we can support the following methods, akin to those provided by a `PositionalList` (Section 7.4.1).

first( ): Return the position containing the least key, or `None` if the tree is empty.

last( ): Return the position containing the greatest key, or `None` if empty tree.

before(p): Return the position containing the greatest key that is less than that of position `p` (i.e., the position that would be visited immediately before `p` in an inorder traversal), or `None` if `p` is the first position.

after(p): Return the position containing the least key that is greater than that of position `p` (i.e., the position that would be visited immediately after `p` in an inorder traversal), or `None` if `p` is the last position.

The "first" position of a binary search tree can be located by starting a walk at the root and continuing to the left child, as long as a left child exists. By symmetry, the last position is reached by repeated steps rightward starting at the root.

The successor of a position, after(p), is determined by the following algorithm.

```
Algorithm after(p):
  if right(p) is not None then {successor is leftmost position in p's right subtree}
    walk = right(p)
    while left(walk) is not None do
      walk = left(walk)
    return walk
  else {successor is nearest ancestor having p in its left subtree}
    walk = p
    ancestor = parent(walk)
    while ancestor is not None and walk == right(ancestor) do
      walk = ancestor
      ancestor = parent(walk)
    return ancestor
```

Code Fragment 11.1: Computing the successor of a position in a binary search tree.

The rationale for this process is based purely on the workings of an inorder traversal, given the correspondence of Proposition 11.1. If p has a right subtree, that right subtree is recursively traversed immediately after p is visited, and so the first position to be visited after p is the *leftmost* position within the right subtree. If p does not have a right subtree, then the flow of control of an inorder traversal returns to p's parent. If p were in the *right* subtree of that parent, then the parent's subtree traversal is complete and the flow of control progresses to its parent and so on. Once an ancestor is reached in which the recursion is returning from its *left* subtree, then that ancestor becomes the next position visited by the inorder traversal, and thus is the successor of p. Notice that the only case in which no such ancestor is found is when p was the rightmost (last) position of the full tree, in which case there is no successor.

A symmetric algorithm can be defined to determine the predecessor of a position, before(p). At this point, we note that the running time of single call to after(p) or before(p) is bounded by the height h of the full tree, because it is found after either a single downward walk or a single upward walk. While the worst-case running time is $O(h)$, we note that either of these methods run in $O(1)$ ***amortized*** time, in that series of n calls to after(p) starting at the first position will execute in a total of $O(n)$ time. We leave a formal justification of this fact to Exercise C-11.34, but intuitively the upward and downward paths mimic steps of the inorder traversal (a related argument was made in the justification of Proposition 9.3).

11.1.2 Searches

The most important consequence of the structural property of a binary search tree is its namesake search algorithm. We can attempt to locate a particular key in a binary search tree by viewing it as a decision tree (recall Figure 8.7). In this case, the question asked at each position p is whether the desired key k is less than, equal to, or greater than the key stored at position p, which we denote as p.key( ). If the answer is "less than," then the search continues in the left subtree. If the answer is "equal," then the search terminates successfully. If the answer is "greater than," then the search continues in the right subtree. Finally, if we reach an empty subtree, then the search terminates unsuccessfully. (See Figure 11.2.)

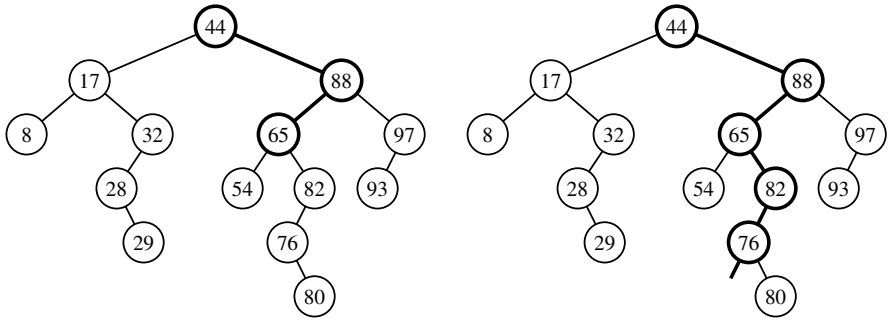


Figure 11.2: (a) A successful search for key 65 in a binary search tree; (b) an unsuccessful search for key 68 that terminates because there is no subtree to the left of the key 76.

We describe this approach in Code Fragment 11.2. If key k occurs in a subtree rooted at p, a call to TreeSearch(T, p, k) results in the position at which the key is found; in this case, the __getitem__ map operation would return the associated value at that position. In the event of an unsuccessful search, the TreeSearch algorithm returns the final position explored on the search path (which we will later make use of when determining where to insert a new item in a search tree).

Algorithm TreeSearch(T, p, k):
  if k == p.key() **then**
    return *p* {successful search}
  else if k < p.key() and T.left(p) is not None **then**
    return TreeSearch(T, T.left(p), k) {recur on left subtree}
  else if k > p.key() and T.right(p) is not None **then**
    return TreeSearch(T, T.right(p), k) {recur on right subtree}
  return p {unsuccessful search}

Code Fragment 11.2: Recursive search in a binary search tree.

Analysis of Binary Tree Searching

The analysis of the worst-case running time of searching in a binary search tree T is simple. Algorithm TreeSearch is recursive and executes a constant number of primitive operations for each recursive call. Each recursive call of TreeSearch is made on a child of the previous position. That is, TreeSearch is called on the positions of a path of T that starts at the root and goes down one level at a time. Thus, the number of such positions is bounded by $h+1$, where h is the height of T. In other words, since we spend $O(1)$ time per position encountered in the search, the overall search runs in $O(h)$ time, where h is the height of the binary search tree T. (See Figure 11.3.)

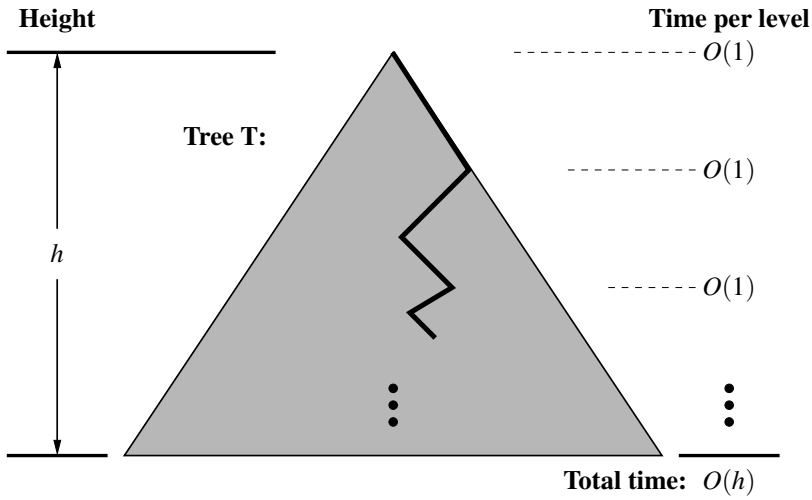


Figure 11.3: Illustrating the running time of searching in a binary search tree. The figure uses standard caricature of a binary search tree as a big triangle and a path from the root as a zig-zag line.

In the context of the sorted map ADT, the search will be used as a subroutine for implementing the __getitem__ method, as well as for the __setitem__ and __delitem__ methods, since each of these begins by trying to locate an existing item with a given key. To implement sorted map operations such as find_lt and find_gt, we will combine this search with traversal methods before and after. All of these operations will run in worst-case $O(h)$ time for a tree with height h. We can use a variation of this technique to implement the find_range method in time $O(s+h)$, where s is the number of items reported (see Exercise C-11.34).

Admittedly, the height h of T can be as large as the number of entries, n, but we expect that it is usually much smaller. Indeed, later in this chapter we show various strategies to maintain an upper bound of $O(\log n)$ on the height of a search tree T.

11.1.3 Insertions and Deletions

Algorithms for inserting or deleting entries of a binary search tree are fairly straightforward, although not trivial.

Insertion

The map command M[k] = v, as supported by the __setitem__ method, begins with a search for key k (assuming the map is nonempty). If found, that item's existing value is reassigned. Otherwise, a node for the new item can be inserted into the underlying tree T in place of the empty subtree that was reached at the end of the failed search. The binary search tree property is sustained by that placement (note that it is placed exactly where a search would expect it). Pseudo-code for such a TreeInsert algorithm is given in in Code Fragment 11.3.

Algorithm TreeInsert(T, k, v):
Input: A search key k to be associated with value v
p = TreeSearch(T, T.root(), k)
if k == p.key() **then**
  Set p's value to v
else if k < p.key() **then**
  add node with item (k,v) as left child of p
else
  add node with item (k,v) as right child of p

Code Fragment 11.3: Algorithm for inserting a key-value pair into a map that is represented as a binary search tree.

An example of insertion into a binary search tree is shown in Figure 11.4.

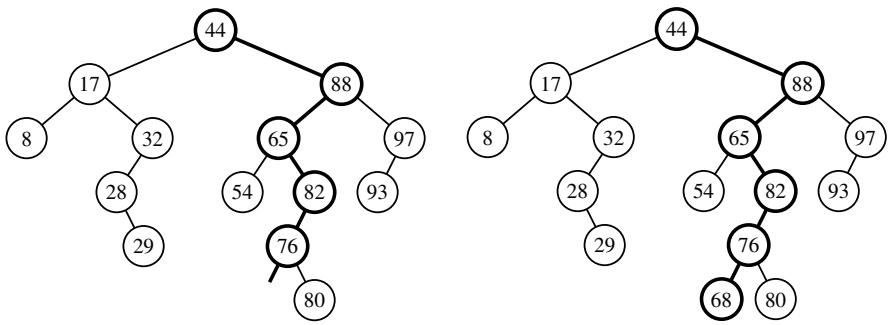


Figure 11.4: Insertion of an item with key 68 into the search tree of Figure 11.2. Finding the position to insert is shown in (a), and the resulting tree is shown in (b).

Deletion

Deleting an item from a binary search tree T is a bit more complex than inserting a new item because the location of the deletion might be anywhere in the tree. (In contrast, insertions are always enacted at the bottom of a path.) To delete an item with key k, we begin by calling TreeSearch(T, T.root( ), k) to find the position p of T storing an item with key equal to k. If the search is successful, we distinguish between two cases (of increasing difficulty):

- If p has at most one child, the deletion of the node at position p is easily implemented. When introducing update methods for the LinkedBinaryTree class in Section 8.3.1, we declared a nonpublic utility, _delete(p), that deletes a node at position p and replaces it with its child (if any), presuming that p has at most one child. That is precisely the desired behavior. It removes the item with key k from the map while maintaining all other ancestor-descendant relationships in the tree, thereby assuring the upkeep of the binary search tree property. (See Figure 11.5.)
- If position p has two children, we cannot simply remove the node from T since this would create a "hole" and two orphaned children. Instead, we proceed as follows (see Figure 11.6):
  - We locate position r containing the item having the greatest key that is strictly less than that of position p, that is, r = before(p) by the notation of Section 11.1.1. Because p has two children, its predecessor is the rightmost position of the left subtree of p.
  - We use r's item as a replacement for the one being deleted at position p. Because r has the immediately preceding key in the map, any items in p's right subtree will have keys greater than r and any other items in p's left subtree will have keys less than r. Therefore, the binary search tree property is satisfied after the replacement.
  - Having used r's as a replacement for p, we instead delete the node at position r from the tree. Fortunately, since r was located as the rightmost position in a subtree, r does not have a right child. Therefore, its deletion can be performed using the first (and simpler) approach.

As with searching and insertion, this algorithm for a deletion involves the traversal of a single path downward from the root, possibly moving an item between two positions of this path, and removing a node from that path and promoting its child. Therefore, it executes in time $O(h)$ where h is the height of the tree.

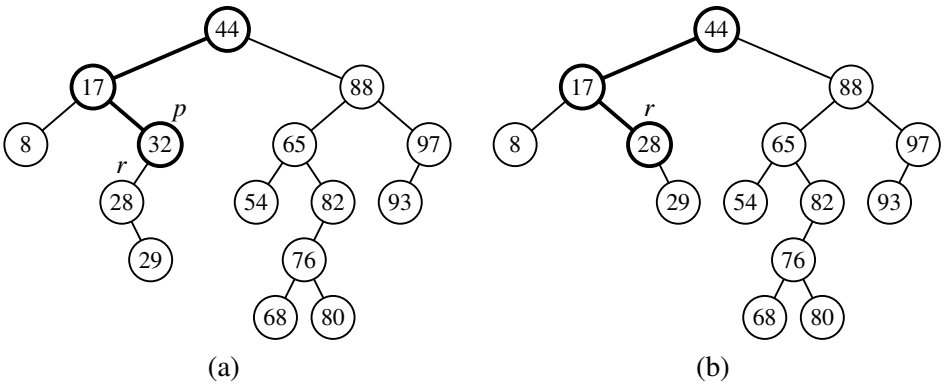


Figure 11.5: Deletion from the binary search tree of Figure 11.4b, where the item to delete (with key 32) is stored at a position p with one child r: (a) before the deletion; (b) after the deletion.

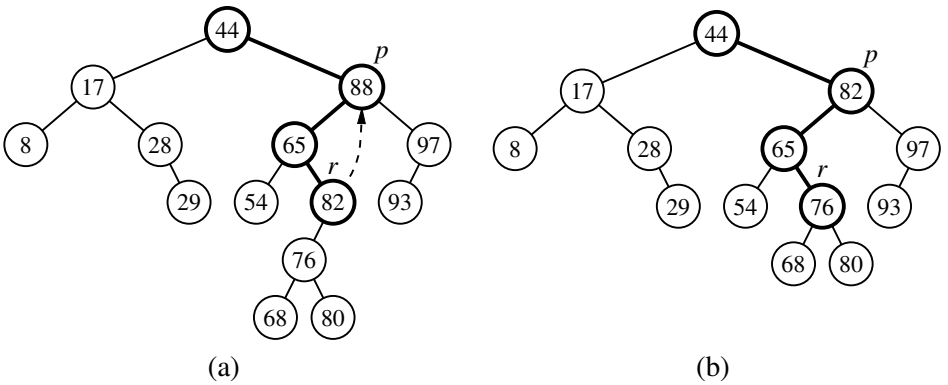


Figure 11.6: Deletion from the binary search tree of Figure 11.5b, where the item to delete (with key 88) is stored at a position p with two children, and replaced by its predecessor r: (a) before the deletion; (b) after the deletion.

11.1.4 Python Implementation

In Code Fragments 11.4 through 11.8 we define a TreeMap class that implements the sorted map ADT using a binary search tree. In fact, our implementation is more general. We support all of the standard map operations (Section 10.1.1), all additional sorted map operations (Section 10.3), and positional operations including first( ), last( ), find_position(k), before(p), after(p), and delete(p).

Our TreeMap class takes advantage of ***multiple inheritance*** for code reuse, inheriting from the LinkedBinaryTree class of Section 8.3.1 for our representation as a positional binary tree, and from the MapBase class from Code Fragment 10.2 of Section 10.1.4 to provide us with the key-value composite item and the concrete behaviors from the collections.MutableMapping abstract base class. We subclass the nested Position class to support more specific p.key( ) and p.value( ) accessors for our map, rather than the p.element( ) syntax inherited from the tree ADT.

We define several nonpublic utilities, most notably a _subtree_search(p, k) method that corresponds to the TreeSearch algorithm of Code Fragment 11.2. That returns a position, ideally one that contains the key k, or otherwise the last position that is visited on the search path. We rely on the fact that the final position during an unsuccessful search is either the nearest key less than k or the nearest key greater than k. This search utility becomes the basis for the public find_position(k) method, and also for internal use when searching, inserting, or deleting items from a map, as well as for the robust searches of the sorted map ADT.

When making structural modifications to the tree, we rely on nonpublic update methods, such as _add_right, that are inherited from the LinkedBinaryTree class (see Section 8.3.1). It is important that these inherited methods remain nonpublic, as the search tree property could be violated through misuse of such operations.

Finally, we note that our code is peppered with calls to presumed methods named _rebalance_insert, _rebalance_delete, and _rebalance_access. These methods serve as ***hooks*** for future use when balancing search trees; we discuss them in Section 11.2. We conclude with a brief guide to the organization of our code.

Code Fragment 11.4: Beginning of TreeMap class including redefined Position class and nonpublic search utilities.

Code Fragment 11.5: Positional methods first( ), last( ), before(p), after(p), and find_position(p) accessor.

Code Fragment 11.6: Selected methods of the sorted map ADT: find_min( ), find_ge(k), and find_range(start, stop); related methods are omitted for the sake of brevity.

Code Fragment 11.7: __getitem__(k), __setitem__(k, v), and __iter__( ).

Code Fragment 11.8: Deletion either by position, as delete(p), or by key, as __delitem__(k).

```
class TreeMap(LinkedBinaryTree, MapBase):
  """Sorted map implementation using a binary search tree."""

  #---------------------------- override Position class ----------------------------
  class Position(LinkedBinaryTree.Position):
    def key(self):
      """Return key of map's key-value pair."""
      return self.element()._key

    def value(self):
      """Return value of map's key-value pair."""
      return self.element()._value

  #------------------------------ nonpublic utilities ------------------------------
  def _subtree_search(self, p, k):
    """Return Position of p's subtree having key k, or last node searched."""
    if k == p.key():                                  # found match
      return p
    elif k < p.key():                                 # search left subtree
      if self.left(p) is not None:
        return self._subtree_search(self.left(p), k)
    else:                                             # search right subtree
      if self.right(p) is not None:
        return self._subtree_search(self.right(p), k)
    return p                                          # unsucessful search

  def _subtree_first_position(self, p):
    """Return Position of first item in subtree rooted at p."""
    walk = p
    while self.left(walk) is not None:                # keep walking left
      walk = self.left(walk)
    return walk

  def _subtree_last_position(self, p):
    """Return Position of last item in subtree rooted at p."""
    walk = p
    while self.right(walk) is not None:               # keep walking right
      walk = self.right(walk)
    return walk
```

Code Fragment 11.4: Beginning of a TreeMap class based on a binary search tree.

```
40    def first(self):
41      """Return the first Position in the tree (or None if empty)."""
42      return self._subtree_first_position(self.root( )) if len(self) > 0 else None
43
44    def last(self):
45      """Return the last Position in the tree (or None if empty)."""
46      return self._subtree_last_position(self.root( )) if len(self) > 0 else None
47
48    def before(self, p):
49      """Return the Position just before p in the natural order.
50
51      Return None if p is the first position.
52      """
53      self._validate(p)                       # inherited from LinkedBinaryTree
54      if self.left(p):
55        return self._subtree_last_position(self.left(p))
56      else:
57        # walk upward
58        walk = p
59        above = self.parent(walk)
60        while above is not None and walk == self.left(above):
61          walk = above
62          above = self.parent(walk)
63        return above
64
65    def after(self, p):
66      """Return the Position just after p in the natural order.
67
68      Return None if p is the last position.
69      """
70      # symmetric to before(p)
71
72    def find_position(self, k):
73      """Return position with key k, or else neighbor (or None if empty)."""
74      if self.is_empty( ):
75        return None
76      else:
77        p = self._subtree_search(self.root( ), k)
78        self._rebalance_access(p)             # hook for balanced tree subclasses
79        return p
```

Code Fragment 11.5: Navigational methods of the TreeMap class.

```
80    def find_min(self):
81      """Return (key,value) pair with minimum key (or None if empty)."""
82      if self.is_empty( ):
83        return None
84      else:
85        p = self.first( )
86        return (p.key( ), p.value( ))
87
88    def find_ge(self, k):
89      """Return (key,value) pair with least key greater than or equal to k.
90
91      Return None if there does not exist such a key.
92      """
93      if self.is_empty( ):
94        return None
95      else:
96        p = self.find_position(k)                    # may not find exact match
97        if p.key( ) < k:                             # p's key is too small
98          p = self.after(p)
99        return (p.key( ), p.value( )) if p is not None else None
100
101   def find_range(self, start, stop):
102     """Iterate all (key,value) pairs such that start <= key < stop.
103
104     If start is None, iteration begins with minimum key of map.
105     If stop is None, iteration continues through the maximum key of map.
106     """
107     if not self.is_empty( ):
108       if start is None:
109         p = self.first( )
110       else:
111         # we initialize p with logic similar to find_ge
112         p = self.find_position(start)
113         if p.key( ) < start:
114           p = self.after(p)
115       while p is not None and (stop is None or p.key( ) < stop):
116         yield (p.key( ), p.value( ))
117         p = self.after(p)
```

Code Fragment 11.6: Some of the sorted map operations for the TreeMap class.

```
118   def __getitem__(self, k):
119     """Return value associated with key k (raise KeyError if not found)."""
120     if self.is_empty( ):
121       raise KeyError('Key Error: ' + repr(k))
122     else:
123       p = self._subtree_search(self.root( ), k)
124       self._rebalance_access(p)            # hook for balanced tree subclasses
125       if k != p.key( ):
126         raise KeyError('Key Error: ' + repr(k))
127       return p.value( )
128
129   def __setitem__(self, k, v):
130     """Assign value v to key k, overwriting existing value if present."""
131     if self.is_empty( ):
132       leaf = self._add_root(self._Item(k,v))          # from LinkedBinaryTree
133     else:
134       p = self._subtree_search(self.root( ), k)
135       if p.key( ) == k:
136         p.element( )._value = v              # replace existing item's value
137         self._rebalance_access(p)            # hook for balanced tree subclasses
138         return
139       else:
140         item = self._Item(k,v)
141         if p.key( ) < k:
142           leaf = self._add_right(p, item)  # inherited from LinkedBinaryTree
143         else:
144           leaf = self._add_left(p, item)   # inherited from LinkedBinaryTree
145     self._rebalance_insert(leaf)             # hook for balanced tree subclasses
146
147   def __iter__(self):
148     """Generate an iteration of all keys in the map in order."""
149     p = self.first( )
150     while p is not None:
151       yield p.key( )
152       p = self.after(p)
```

Code Fragment 11.7: Map operations for accessing and inserting items in the TreeMap class. Reverse iteration can be implemented with __reverse__, using symmetric approach to __iter__.

```
153   def delete(self, p):
154     """Remove the item at given Position."""
155     self._validate(p)                          # inherited from LinkedBinaryTree
156     if self.left(p) and self.right(p):         # p has two children
157       replacement = self._subtree_last_position(self.left(p))
158       self._replace(p, replacement.element())  # from LinkedBinaryTree
159       p = replacement
160     # now p has at most one child
161     parent = self.parent(p)
162     self._delete(p)                            # inherited from LinkedBinaryTree
163     self._rebalance_delete(parent)             # if root deleted, parent is None
164
165   def __delitem__(self, k):
166     """Remove item associated with key k (raise KeyError if not found)."""
167     if not self.is_empty():
168       p = self._subtree_search(self.root(), k)
169       if k == p.key():
170         self.delete(p)                         # rely on positional version
171         return                                 # successful deletion complete
172       self._rebalance_access(p)                # hook for balanced tree subclasses
173     raise KeyError('Key Error: ' + repr(k))
```

Code Fragment 11.8: Support for deleting an item from a TreeMap, located either by position or by key.

11.1.5 Performance of a Binary Search Tree

An analysis of the operations of our TreeMap class is given in Table 11.1. Almost all operations have a worst-case running time that depends on h, where h is the height of the current tree. This is because most operations rely on a constant amount of work for each node along a particular path of the tree, and the maximum path length within a tree is proportional to the height of the tree. Most notably, our implementations of map operations __getitem__, __setitem__, and __delitem__ each begin with a call to the _subtree_search utility which traces a path downward from the root of the tree, using $O(1)$ time at each node to determine how to continue the search. Similar paths are traced when looking for a replacement during a deletion, or when computing a position's inorder predecessor or successor. We note that although a single call to the after method has worst-case running time of $O(h)$, the n successive calls made during a call to __iter__ require a total of $O(n)$ time, since each edge is traced at most twice; in a sense, those calls have $O(1)$ amortized time bounds. A similar argument can be used to prove the $O(s+h)$ worst-case bound for a call to find_range that reports s results (see Exercise C-11.34).

Operation	Running Time
k in T	$O(h)$
T[k], T[k] = v	$O(h)$
T.delete(p), del T[k]	$O(h)$
T.find_position(k)	$O(h)$
T.first( ), T.last( ), T.find_min( ), T.find_max( )	$O(h)$
T.before(p), T.after(p)	$O(h)$
T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k)	$O(h)$
T.find_range(start, stop)	$O(s+h)$
iter(T), reversed(T)	$O(n)$

Table 11.1: Worst-case running times of the operations for a TreeMap T. We denote the current height of the tree with h, and the number of items reported by find_range as s. The space usage is $O(n)$, where n is the number of items stored in the map.

A binary search tree T is therefore an efficient implementation of a map with n entries only if its height is small. In the best case, T has height $h = \lceil \log(n+1) \rceil - 1$, which yields logarithmic-time performance for all the map operations. In the worst case, however, T has height n, in which case it would look and feel like an ordered list implementation of a map. Such a worst-case configuration arises, for example, if we insert items with keys in increasing or decreasing order. (See Figure 11.7.)

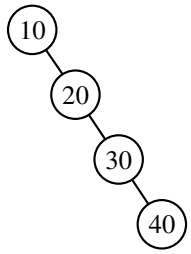


Figure 11.7: Example of a binary search tree with linear height, obtained by inserting entries with keys in increasing order.

We can nevertheless take comfort that, on average, a binary search tree with n keys generated from a random series of insertions and removals of keys has expected height $O(\log n)$; the justification of this statement is beyond the scope of the book, requiring careful mathematical language to precisely define what we mean by a random series of insertions and removals, and sophisticated probability theory.

In applications where one cannot guarantee the random nature of updates, it is better to rely on variations of search trees, presented in the remainder of this chapter, that guarantee a ***worst-case*** height of $O(\log n)$, and thus $O(\log n)$ worst-case time for searches, insertions, and deletions.

11.2 Balanced Search Trees

In the closing of the previous section, we noted that if we could assume a random series of insertions and removals, the standard binary search tree supports $O(\log n)$ expected running times for the basic map operations. However, we may only claim $O(n)$ worst-case time, because some sequences of operations may lead to an unbalanced tree with height proportional to n.

In the remainder of this chapter, we explore four search tree algorithms that provide stronger performance guarantees. Three of the four data structures (AVL trees, splay trees, and red-black trees) are based on augmenting a standard binary search tree with occasional operations to reshape the tree and reduce its height.

The primary operation to rebalance a binary search tree is known as a ***rotation***. During a rotation, we "rotate" a child to be above its parent, as diagrammed in Figure 11.8.

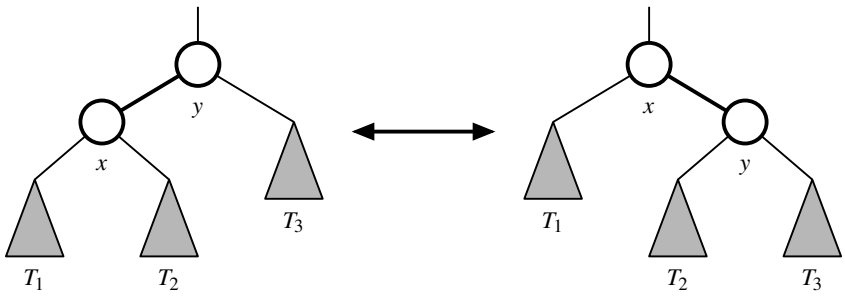


Figure 11.8: A rotation operation in a binary search tree. A rotation can be performed to transform the left formation into the right, or the right formation into the left. Note that all keys in subtree T_1 have keys less than that of position x, all keys in subtree T_2 have keys that are between those of positions x and y, and all keys in subtree T_3 have keys that are greater than that of position y.

To maintain the binary search tree property through a rotation, we note that if position x was a left child of position y prior to a rotation (and therefore the key of x is less than the key of y), then y becomes the *right* child of x after the rotation, and vice versa. Furthermore, we must relink the subtree of items with keys that lie between the keys of the two positions that are being rotated. For example, in Figure 11.8 the subtree labeled T_2 represents items with keys that are known to be greater than that of position x and less than that of position y. In the first configuration of that figure, T_2 is the right subtree of position x; in the second configuration, it is the left subtree of position y.

Because a single rotation modifies a constant number of parent-child relationships, it can be implemented in $O(1)$ time with a linked binary tree representation.

In the context of a tree-balancing algorithm, a rotation allows the shape of a tree to be modified while maintaining the search tree property. If used wisely, this operation can be performed to avoid highly unbalanced tree configurations. For example, a rightward rotation from the first formation of Figure 11.8 to the second reduces the depth of each node in subtree T_1 by one, while increasing the depth of each node in subtree T_3 by one. (Note that the depth of nodes in subtree T_2 are unaffected by the rotation.)

One or more rotations can be combined to provide broader rebalancing within a tree. One such compound operation we consider is a ***trinode restructuring***. For this manipulation, we consider a position x, its parent y, and its grandparent z. The goal is to restructure the subtree rooted at z in order to reduce the overall path length to x and its subtrees. Pseudo-code for a restructure(x) method is given in Code Fragment 11.9 and illustrated in Figure 11.9. In describing a trinode restructuring, we temporarily rename the positions x, y, and z as a, b, and c, so that a precedes b and b precedes c in an inorder traversal of T. There are four possible orientations mapping x, y, and z to a, b, and c, as shown in Figure 11.9, which are unified into one case by our relabeling. The trinode restructuring replaces z with the node identified as b, makes the children of this node be a and c, and makes the children of a and c be the four previous children of x, y, and z (other than x and y), while maintaining the inorder relationships of all the nodes in T.

Algorithm restructure(x):

Input: A position x of a binary search tree T that has both a parent y and a grandparent z

Output: Tree T after a trinode restructuring (which corresponds to a single or double rotation) involving positions x, y, and z

1: Let (a, b, c) be a left-to-right (inorder) listing of the positions x, y, and z, and let (T_1, T_2, T_3, T_4) be a left-to-right (inorder) listing of the four subtrees of x, y, and z not rooted at x, y, or z.
2: Replace the subtree rooted at z with a new subtree rooted at b.
3: Let a be the left child of b and let T_1 and T_2 be the left and right subtrees of a, respectively.
4: Let c be the right child of b and let T_3 and T_4 be the left and right subtrees of c, respectively.

Code Fragment 11.9: The trinode restructuring operation in a binary search tree.

In practice, the modification of a tree T caused by a trinode restructuring operation can be implemented through case analysis either as a single rotation (as in Figure 11.9a and b) or as a double rotation (as in Figure 11.9c and d). The double rotation arises when position x has the middle of the three relevant keys and is first rotated above its parent, and then above what was originally its grandparent. In any of the cases, the trinode restructuring is completed with $O(1)$ running time.

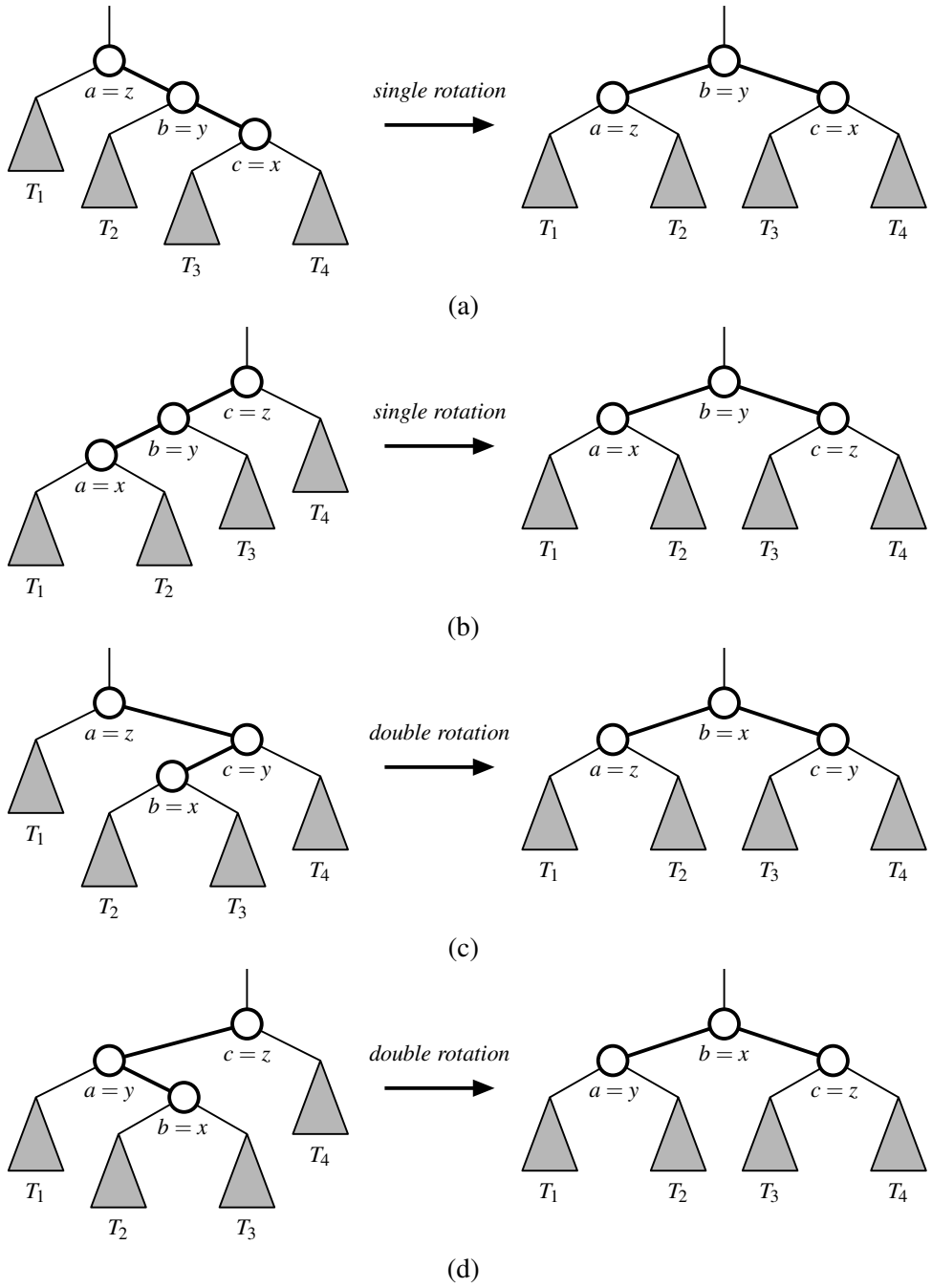


Figure 11.9: Schematic illustration of a trinode restructuring operation: (a and b) require a single rotation; (c and d) require a double rotation.

11.2.1 Python Framework for Balancing Search Trees

Our TreeMap class, introduced in Section 11.1.4, is a concrete map implementation that does not perform any explicit balancing operations. However, we designed that class to also serve as a base class for other subclasses that implement more advanced tree-balancing algorithms. A summary of our inheritance hierarchy is shown in Figure 11.10.

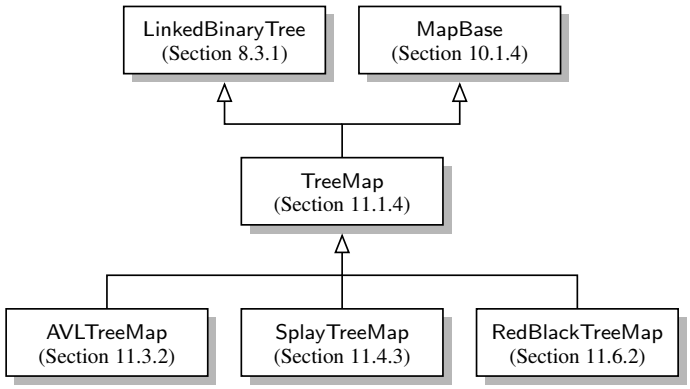


Figure 11.10: Our hierarchy of balanced search trees (with references to where they are defined). Recall that TreeMap inherits multiply from LinkedBinaryTree and MapBase.

Hooks for Rebalancing Operations

Our implementation of the basic map operations in Section 11.1.4 includes strategic calls to three nonpublic methods that serve as ***hooks*** for rebalancing algorithms:

- A call to _rebalance_insert(p) is made from within the __setitem__ method immediately after a new node is added to the tree at position *p*.
- A call to _rebalance_delete(p) is made each time a node has been deleted from the tree, with position *p* identifying the *parent* of the node that has just been removed. Formally, this hook is called from within the public delete(p) method, which is indirectly invoked by the public __delitem__(k) behavior.
- We also provide a hook, _rebalance_access(p), that is called when an item at position *p* of a tree is accessed through a public method such as __getitem__. This hook is used by the ***splay tree*** structure (see Section 11.4) to restructure a tree so that more frequently accessed items are brought closer to the root.

We provide trivial declarations of these three methods, in Code Fragment 11.10, having bodies that do nothing (using the **pass** statement). A subclass of TreeMap may override any of these methods to implement a nontrivial action to rebalance a tree. This is another example of the ***template method design pattern***, as seen in Section 8.4.6.

```
174    def _rebalance_insert(self, p): pass
175    def _rebalance_delete(self, p): pass
176    def _rebalance_access(self, p): pass
```

Code Fragment 11.10: Additional code for the TreeMap class (continued from Code Fragment 11.8), providing stubs for the rebalancing hooks.

Nonpublic Methods for Rotating and Restructuring

A second form of support for balanced search trees is our inclusion of nonpublic utility methods _rotate and _restructure that, respectively, implement a single rotation and a trinode restructuring (described at the beginning of Section 11.2). Although these methods are not invoked by the public TreeMap operations, we promote code reuse by providing these implementation in this class so that they are inherited by all balanced-tree subclasses.

Our implementations are provided in Code Fragment 11.11. To simplify the code, we define an additional _relink utility that properly links parent and child nodes to each other, including the special case in which a "child" is a None reference. The focus of the _rotate method then becomes redefining the relationship between the parent and child, relinking a rotated node directly to its original grandparent, and shifting the "middle" subtree (that labeled as T_2 in Figure 11.8) between the rotated nodes. For the trinode restructuring, we determine whether to perform a single or double rotation, as originally described in Figure 11.9.

Factory for Creating Tree Nodes

We draw attention to an important subtlety in the design of both our TreeMap class and the original LinkedBinaryTree subclass. The low-level definition of a node is provided by the nested _Node class within LinkedBinaryTree. Yet, several of our tree-balancing strategies require that auxiliary information be stored at each node to guide the balancing process. Those classes will override the nested _Node class to provide storage for an additional field.

Whenever we add a new node to the tree, as within the _add_right method of the LinkedBinaryTree (originally given in Code Fragment 8.10), we intentionally instantiate the node using the syntax self._Node, rather than the qualified name LinkedBinaryTree._Node. This is vital to our framework! When the expression self._Node is applied to an instance of a tree (sub)class, Python's name resolution follows the inheritance structure (as described in Section 2.5.2). If a subclass has overridden the definition for the _Node class, instantiation of self._Node relies on the newly defined node class. This technique is an example of the ***factory method design pattern***, as we provide a *subclass* the means to control the type of node that is created within methods of the *parent* class.

```
177   def _relink(self, parent, child, make_left_child):
178     """Relink parent node with child node (we allow child to be None)."""
179     if make_left_child:                          # make it a left child
180       parent._left = child
181     else:                                        # make it a right child
182       parent._right = child
183     if child is not None:                        # make child point to parent
184       child._parent = parent
185
186   def _rotate(self, p):
187     """Rotate Position p above its parent."""
188     x = p._node
189     y = x._parent                                # we assume this exists
190     z = y._parent                                # grandparent (possibly None)
191     if z is None:
192       self._root = x                             # x becomes root
193       x._parent = None
194     else:
195       self._relink(z, x, y == z._left)           # x becomes a direct child of z
196     # now rotate x and y, including transfer of middle subtree
197     if x == y._left:
198       self._relink(y, x._right, True)            # x._right becomes left child of y
199       self._relink(x, y, False)                  # y becomes right child of x
200     else:
201       self._relink(y, x._left, False)            # x._left becomes right child of y
202       self._relink(x, y, True)                   # y becomes left child of x
203
204   def _restructure(self, x):
205     """Perform trinode restructure of Position x with parent/grandparent."""
206     y = self.parent(x)
207     z = self.parent(y)
208     if (x == self.right(y)) == (y == self.right(z)):  # matching alignments
209       self._rotate(y)                            # single rotation (of y)
210       return y                                   # y is new subtree root
211     else:                                        # opposite alignments
212       self._rotate(x)                            # double rotation (of x)
213       self._rotate(x)
214       return x                                   # x is new subtree root
```

Code Fragment 11.11: Additional code for the TreeMap class (continued from Code Fragment 11.10), to provide nonpublic utilities for balanced search tree subclasses.

11.3 AVL Trees

The TreeMap class, which uses a standard binary search tree as its data structure, should be an efficient map data structure, but its worst-case performance for the various operations is linear time, because it is possible that a series of operations results in a tree with linear height. In this section, we describe a simple balancing strategy that guarantees worst-case logarithmic running time for all the fundamental map operations.

Definition of an AVL Tree

The simple correction is to add a rule to the binary search tree definition that will maintain a logarithmic height for the tree. Although we originally defined the height of a subtree rooted at position p of a tree to be the number of *edges* on the longest path from p to a leaf (see Section 8.1.3), it is easier for explanation in this section to consider the height to be the number of *nodes* on such a longest path. By this definition, a leaf position has height 1, while we trivially define the height of a "null" child to be 0.

In this section, we consider the following ***height-balance property***, which characterizes the structure of a binary search tree T in terms of the heights of its nodes.

***Height-Balance Property*:** For every position p of T, the heights of the children of p differ by at most 1.

Any binary search tree T that satisfies the height-balance property is said to be an ***AVL tree***, named after the initials of its inventors: Adel'son-Vel'skii and Landis. An example of an AVL tree is shown in Figure 11.11.

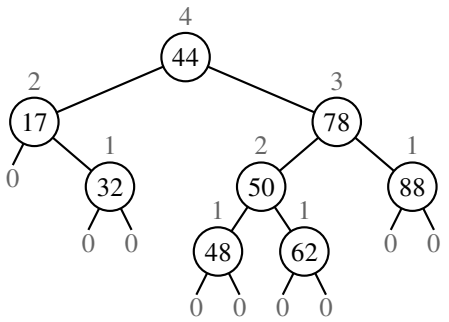


Figure 11.11: An example of an AVL tree. The keys of the items are shown inside the nodes, and the heights of the nodes are shown above the nodes (with empty subtrees having height 0).

An immediate consequence of the height-balance property is that a subtree of an AVL tree is itself an AVL tree. The height-balance property has also the important consequence of keeping the height small, as shown in the following proposition.

Proposition 11.2: *The height of an AVL tree storing n entries is $O(\log n)$.*

Justification: Instead of trying to find an upper bound on the height of an AVL tree directly, it turns out to be easier to work on the "inverse problem" of finding a lower bound on the minimum number of nodes $n(h)$ of an AVL tree with height h. We will show that $n(h)$ grows at least exponentially. From this, it will be an easy step to derive that the height of an AVL tree storing n entries is $O(\log n)$.

We begin by noting that $n(1) = 1$ and $n(2) = 2$, because an AVL tree of height 1 must have exactly one node and an AVL tree of height 2 must have at least two nodes. Now, an AVL tree with the minimum number of nodes having height h for $h \geq 3$, is such that both its subtrees are AVL trees with the minimum number of nodes: one with height $h-1$ and the other with height $h-2$. Taking the root into account, we obtain the following formula that relates $n(h)$ to $n(h-1)$ and $n(h-2)$, for $h \geq 3$:

$$n(h) = 1 + n(h-1) + n(h-2). \tag{11.1}$$

At this point, the reader familiar with the properties of Fibonacci progressions (Section 1.8 and Exercise C-3.49) will already see that $n(h)$ is a function exponential in h. To formalize that observation, we proceed as follows.

Formula 11.1 implies that $n(h)$ is a strictly increasing function of h. Thus, we know that $n(h-1) > n(h-2)$. Replacing $n(h-1)$ with $n(h-2)$ in Formula 11.1 and dropping the 1, we get, for $h \geq 3$,

$$n(h) \; > \; 2 \cdot n(h-2). \tag{11.2}$$

Formula 11.2 indicates that $n(h)$ at least doubles each time h increases by 2, which intuitively means that $n(h)$ grows exponentially. To show this fact in a formal way, we apply Formula 11.2 repeatedly, yielding the following series of inequalities:

$$\begin{aligned} n(h) &> 2 \cdot n(h-2) \\ &> 4 \cdot n(h-4) \\ &> 8 \cdot n(h-6) \\ &\;\;\vdots \\ &> 2^i \cdot n(h-2i). \end{aligned} \tag{11.3}$$

That is, $n(h) > 2^i \cdot n(h-2i)$, for any integer i, such that $h-2i \geq 1$. Since we already know the values of $n(1)$ and $n(2)$, we pick i so that $h-2i$ is equal to either 1 or 2. That is, we pick

$$i = \left\lceil \frac{h}{2} \right\rceil - 1.$$

By substituting the above value of i in Formula 11.3, we obtain, for $h \geq 3$,

$$\begin{aligned} n(h) &> 2^{\lceil \frac{h}{2} \rceil - 1} \cdot n\left(h - 2\left\lceil \frac{h}{2} \right\rceil + 2\right) \\ &\geq 2^{\lceil \frac{h}{2} \rceil - 1} n(1) \\ &\geq 2^{\frac{h}{2}-1}. \end{aligned} \tag{11.4}$$

By taking logarithms of both sides of Formula 11.4, we obtain

$$\log(n(h)) > \frac{h}{2} - 1,$$

from which we get

$$h < 2\log(n(h)) + 2, \tag{11.5}$$

which implies that an AVL tree storing n entries has height at most $2\log n + 2$. ■

By Proposition 11.2 and the analysis of binary search trees given in Section 11.1, the operation `__getitem__`, in a map implemented with an AVL tree, runs in time $O(\log n)$, where n is the number of items in the map. Of course, we still have to show how to maintain the height-balance property after an insertion or deletion.

11.3.1 Update Operations

Given a binary search tree T, we say that a position is ***balanced*** if the absolute value of the difference between the heights of its children is at most 1, and we say that it is ***unbalanced*** otherwise. Thus, the height-balance property characterizing AVL trees is equivalent to saying that every position is balanced.

The insertion and deletion operations for AVL trees begin similarly to the corresponding operations for (standard) binary search trees, but with post-processing for each operation to restore the balance of any portions of the tree that are adversely affected by the change.

Insertion

Suppose that tree T satisfies the height-balance property, and hence is an AVL tree, prior to the insertion of a new item. An insertion of a new item in a binary search tree, as described in Section 11.1.3, results in a new node at a leaf position p. This action may violate the height-balance property (see, for example, Figure 11.12a), yet the only positions that may become unbalanced are ancestors of p, because those are the only positions whose subtrees have changed. Therefore, let us describe how to restructure T to fix any unbalance that may have occurred.

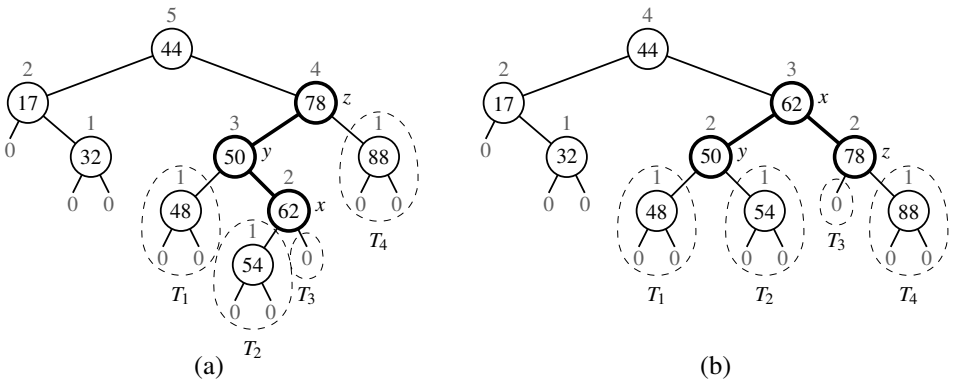


Figure 11.12: An example insertion of an item with key 54 in the AVL tree of Figure 11.11: (a) after adding a new node for key 54, the nodes storing keys 78 and 44 become unbalanced; (b) a trinode restructuring restores the height-balance property. We show the heights of nodes above them, and we identify the nodes x, y, and z and subtrees T_1, T_2, T_3, and T_4 participating in the trinode restructuring.

We restore the balance of the nodes in the binary search tree T by a simple "search-and-repair" strategy. In particular, let z be the first position we encounter in going up from p toward the root of T such that z is unbalanced (see Figure 11.12a.) Also, let y denote the child of z with higher height (and note that y must be an ancestor of p). Finally, let x be the child of y with higher height (there cannot be a tie and position x must also be an ancestor of p, possibly p itself). We rebalance the subtree rooted at z by calling the ***trinode restructuring*** method, `restructure`(x), originally described in Section 11.2. An example of such a restructuring in the context of an AVL insertion is portrayed in Figure 11.12.

To formally argue the correctness of this process in reestablishing the AVL height-balance property, we consider the implication of z being the nearest ancestor of p that became unbalanced after the insertion of p. It must be that the height of y increased by one due to the insertion and that it is now 2 greater than its sibling. Since y remains balanced, it must be that it formerly had subtrees with equal heights, and that the subtree containing x has increased its height by one. That subtree increased either because $x = p$, and thus its height changed from 0 to 1, or because x previously had equal-height subtrees and the height of the one containing p has increased by 1. Letting $h \geq 0$ denote the height of the tallest child of x, this scenario might be portrayed as in Figure 11.13.

After the trinode restructuring, we see that each of x, y, and z has become balanced. Furthermore, the node that becomes the root of the subtree after the restructuring has height $h+2$, which is precisely the height that z had before the insertion of the new item. Therefore, any ancestor of z that became temporarily unbalanced becomes balanced again, and this one restructuring restores the height-balance property ***globally***.

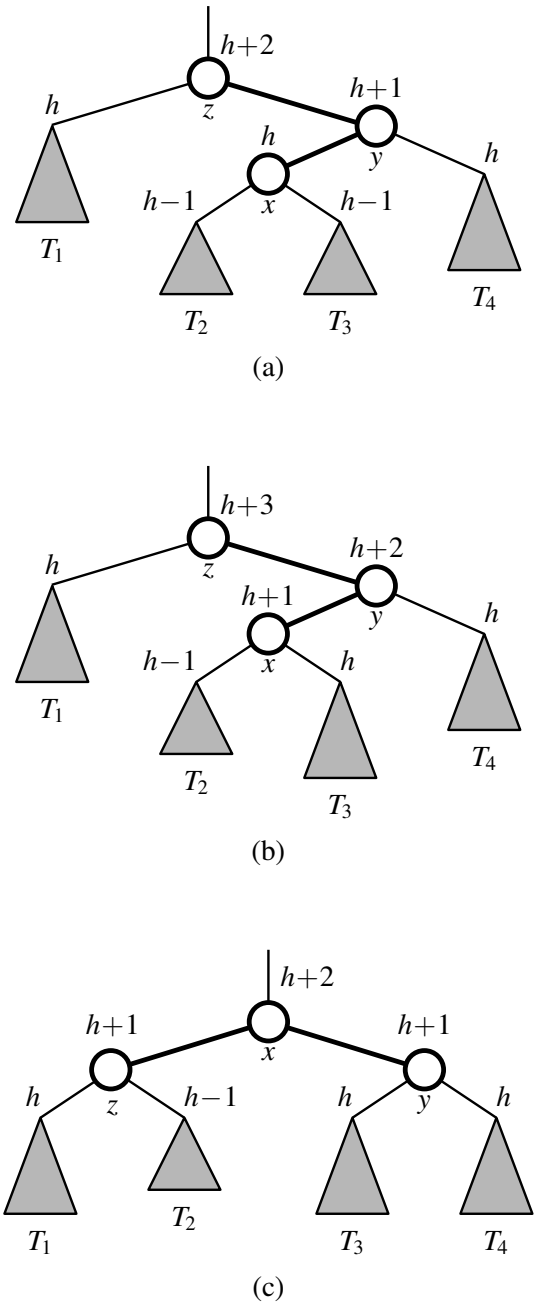


Figure 11.13: Rebalancing of a subtree during a typical insertion into an AVL tree: (a) before the insertion; (b) after an insertion in subtree T_3 causes imbalance at z; (c) after restoring balance with trinode restructuring. Notice that the overall height of the subtree after the insertion is the same as before the insertion.

Deletion

Recall that a deletion from a regular binary search tree results in the structural removal of a node having either zero or one children. Such a change may violate the height-balance property in an AVL tree. In particular, if position p represents the parent of the removed node in tree T, there may be an unbalanced node on the path from p to the root of T. (See Figure 11.14a.) In fact, there can be at most one such unbalanced node. (The justification of this fact is left as Exercise C-11.49.)

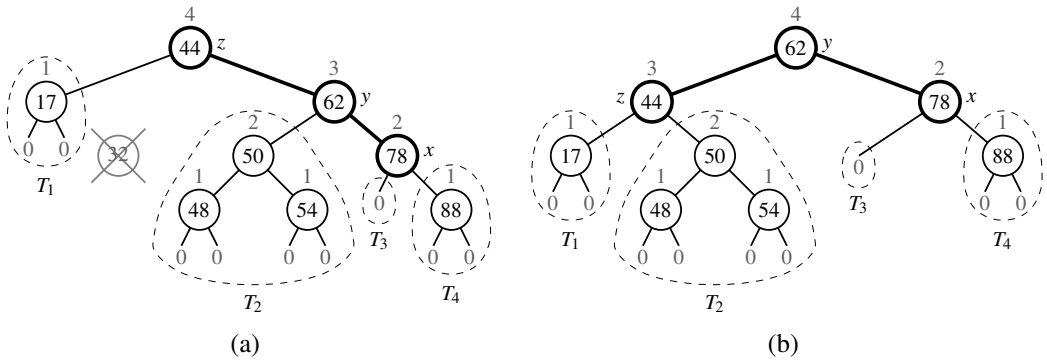


Figure 11.14: Deletion of the item with key 32 from the AVL tree of Figure 11.12b: (a) after removing the node storing key 32, the root becomes unbalanced; (b) a (single) rotation restores the height-balance property.

As with insertion, we use trinode restructuring to restore balance in the tree T. In particular, let z be the first unbalanced position encountered going up from p toward the root of T. Also, let y be the child of z with larger height (note that position y is the child of z that is not an ancestor of p), and let x be the child of y defined as follows: If one of the children of y is taller than the other, let x be the taller child of y; else (both children of y have the same height), let x be the child of y on the same side as y (that is, if y is the left child of z, let x be the left child of y, else let x be the right child of y). In any case, we then perform a $\mathsf{restructure}(x)$ operation. (See Figure 11.14b.)

The restructured subtree is rooted at the middle position denoted as b in the description of the trinode restructuring operation. The height-balance property is guaranteed to be ***locally*** restored within the subtree of b. (See Exercises R-11.11 and R-11.12). Unfortunately, this trinode restructuring may reduce the height of the subtree rooted at b by 1, which may cause an ancestor of b to become unbalanced. So, after rebalancing z, we continue walking up T looking for unbalanced positions. If we find another, we perform a restructure operation to restore its balance, and continue marching up T looking for more, all the way to the root. Still, since the height of T is $O(\log n)$, where n is the number of entries, by Proposition 11.2, $O(\log n)$ trinode restructurings are sufficient to restore the height-balance property.

Performance of AVL Trees

By Proposition 11.2, the height of an AVL tree with n items is guaranteed to be $O(\log n)$. Because the standard binary search tree operation had running times bounded by the height (see Table 11.1), and because the additional work in maintaining balance factors and restructuring an AVL tree can be bounded by the length of a path in the tree, the traditional map operations run in worst-case logarithmic time with an AVL tree. We summarize these results in Table 11.2, and illustrate this performance in Figure 11.15.

Operation	Running Time
k in T	$O(\log n)$
T[k] = v	$O(\log n)$
T.delete(p), del T[k]	$O(\log n)$
T.find_position(k)	$O(\log n)$
T.first( ), T.last( ), T.find_min( ), T.find_max( )	$O(\log n)$
T.before(p), T.after(p)	$O(\log n)$
T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k)	$O(\log n)$
T.find_range(start, stop)	$O(s+\log n)$
iter(T), reversed(T)	$O(n)$

Table 11.2: Worst-case running times of operations for an n-item sorted map realized as an AVL tree T, with s denoting the number of items reported by find_range.

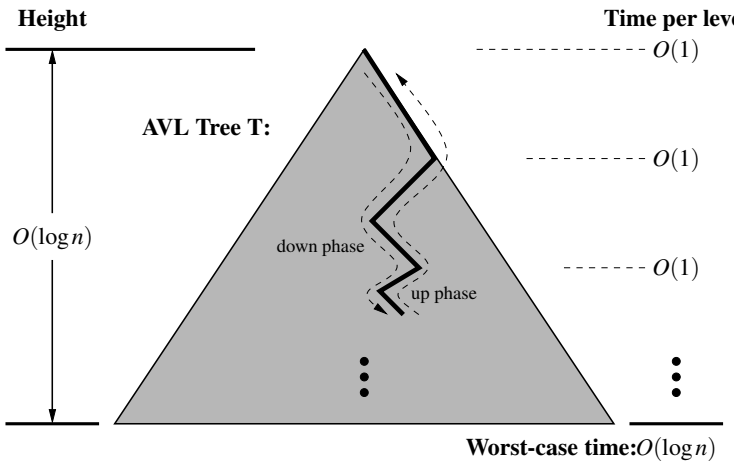


Figure 11.15: Illustrating the running time of searches and updates in an AVL tree. The time performance is $O(1)$ per level, broken into a down phase, which typically involves searching, and an up phase, which typically involves updating height values and performing local trinode restructurings (rotations).

11.3.2 Python Implementation

A complete implementation of an AVLTreeMap class is provided in Code Fragments 11.12 and 11.13. It inherits from the standard TreeMap class and relies on the balancing framework described in Section 11.2.1. We highlight two important aspects of our implementation. First, the AVLTreeMap overrides the definition of the nested _Node class, as shown in Code Fragment 11.12, in order to provide support for storing the height of the subtree stored at a node. We also provide several utilities involving heights of nodes, and the corresponding positions.

To implement the core logic of the AVL balancing strategy, we define a utility, named _rebalance, that suffices as a hook for restoring the height-balance property after an insertion or a deletion. Although the inherited behaviors for insertion and deletion are quite different, the necessary post-processing for an AVL tree can be unified. In both cases, we trace an upward path from the position p at which the change took place, recalculating the height of each position based on the (updated) heights of its children, and using a trinode restructuring operation if an imbalanced position is reached. If we reach an ancestor with height that is unchanged by the overall map operation, or if we perform a trinode restructuring that results in the subtree having the same height it had before the map operation, we stop the process; no further ancestor's height will change. To detect the stopping condition, we record the "old" height of each node and compare it to the newly calculated height.

```
1  class AVLTreeMap(TreeMap):
2    """Sorted map implementation using an AVL tree."""
3
4    #------------------------- nested _Node class -------------------------
5    class _Node(TreeMap._Node):
6      """Node class for AVL maintains height value for balancing."""
7      __slots__ = '_height'          # additional data member to store height
8
9      def __init__(self, element, parent=None, left=None, right=None):
10       super().__init__(element, parent, left, right)
11       self._height = 0               # will be recomputed during balancing
12
13     def left_height(self):
14       return self._left._height if self._left is not None else 0
15
16     def right_height(self):
17       return self._right._height if self._right is not None else 0
```

Code Fragment 11.12: AVLTreeMap class (continued in Code Fragment 11.13).

```
18    #------------------------ positional-based utility methods ------------------------
19    def _recompute_height(self, p):
20      p._node._height = 1 + max(p._node.left_height( ), p._node.right_height( ))
21
22    def _isbalanced(self, p):
23      return abs(p._node.left_height( ) − p._node.right_height( )) <= 1
24
25    def _tall_child(self, p, favorleft=False): # parameter controls tiebreaker
26      if p._node.left_height( ) + (1 if favorleft else 0) > p._node.right_height( ):
27        return self.left(p)
28      else:
29        return self.right(p)
30
31    def _tall_grandchild(self, p):
32      child = self._tall_child(p)
33      # if child is on left, favor left grandchild; else favor right grandchild
34      alignment = (child == self.left(p))
35      return self._tall_child(child, alignment)
36
37    def _rebalance(self, p):
38      while p is not None:
39        old_height = p._node._height          # trivially 0 if new node
40        if not self._isbalanced(p):           # imbalance detected!
41          # perform trinode restructuring, setting p to resulting root,
42          # and recompute new local heights after the restructuring
43          p = self._restructure(self._tall_grandchild(p))
44          self._recompute_height(self.left(p))
45          self._recompute_height(self.right(p))
46        self._recompute_height(p)             # adjust for recent changes
47        if p._node._height == old_height:     # has height changed?
48          p = None                            # no further changes needed
49        else:
50          p = self.parent(p)                  # repeat with parent
51
52    #------------------------- override balancing hooks -------------------------
53    def _rebalance_insert(self, p):
54      self._rebalance(p)
55
56    def _rebalance_delete(self, p):
57      self._rebalance(p)
```

Code Fragment 11.13: AVLTreeMap class (continued from Code Fragment 11.12).

11.4 Splay Trees

The next search tree structure we study is known as a a ***splay tree***. This structure is conceptually quite different from the other balanced search trees we discuss in this chapter, for a splay tree does not strictly enforce a logarithmic upper bound on the height of the tree. In fact, there are no additional height, balance, or other auxiliary data associated with the nodes of this tree.

The efficiency of splay trees is due to a certain move-to-root operation, called ***splaying***, that is performed at the bottommost position p reached during every insertion, deletion, or even a search. (In essence, this is a tree variant of the move-to-front heuristic that we explored for lists in Section 7.6.2.) Intuitively, a splay operation causes more frequently accessed elements to remain nearer to the root, thereby reducing the typical search times. The surprising thing about splaying is that it allows us to guarantee a logarithmic amortized running time, for insertions, deletions, and searches.

11.4.1 Splaying

Given a node x of a binary search tree T, we ***splay*** x by moving x to the root of T through a sequence of restructurings. The particular restructurings we perform are important, for it is not sufficient to move x to the root of T by just any sequence of restructurings. The specific operation we perform to move x up depends upon the relative positions of x, its parent y, and (if it exists) x's grandparent z. There are three cases that we consider.

***zig-zig*:** The node x and its parent y are both left children or both right children. (See Figure 11.16.) We promote x, making y a child of x and z a child of y, while maintaining the inorder relationships of the nodes in T.

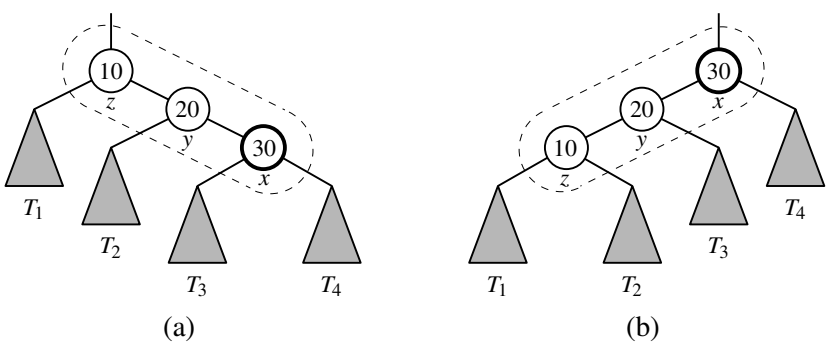


Figure 11.16: Zig-zig: (a) before; (b) after. There is another symmetric configuration where x and y are left children.

***zig-zag*:** One of x and y is a left child and the other is a right child. (See Figure 11.17.) In this case, we promote x by making x have y and z as its children, while maintaining the inorder relationships of the nodes in T.

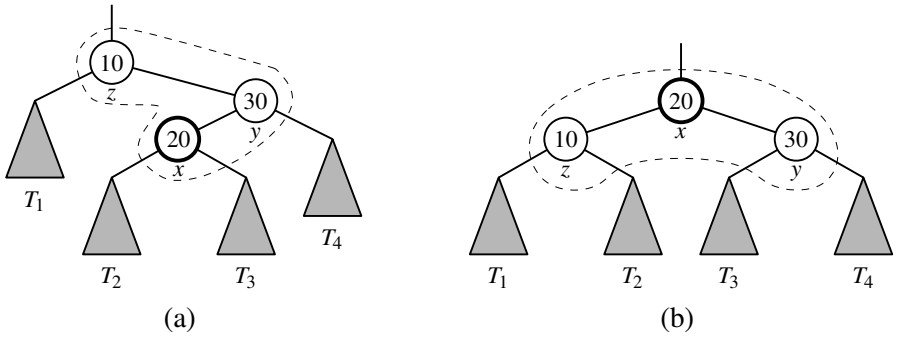


Figure 11.17: Zig-zag: (a) before; (b) after. There is another symmetric configuration where x is a right child and y is a left child.

***zig*:** x does not have a grandparent. (See Figure 11.18.) In this case, we perform a single rotation to promote x over y, making y a child of x, while maintaining the relative inorder relationships of the nodes in T.

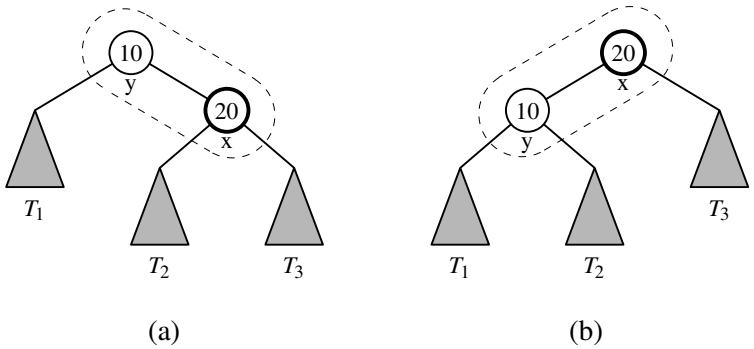
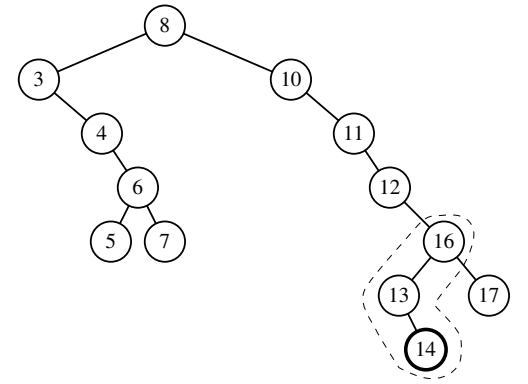
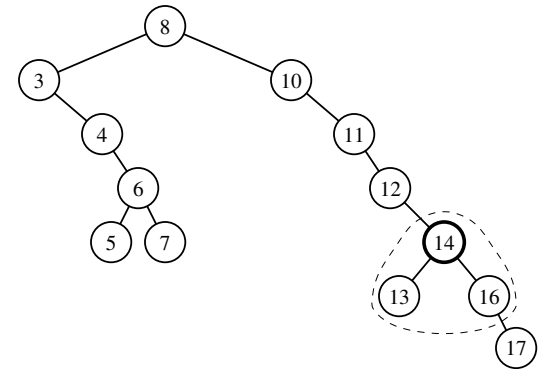


Figure 11.18: Zig: (a) before; (b) after. There is another symmetric configuration where x is originally a left child of y.

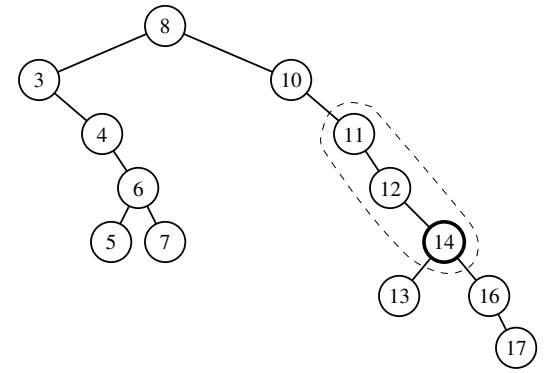
We perform a zig-zig or a zig-zag when x has a grandparent, and we perform a zig when x has a parent but not a grandparent. A ***splaying*** step consists of repeating these restructurings at x until x becomes the root of T. An example of the splaying of a node is shown in Figures 11.19 and 11.20.



(a)

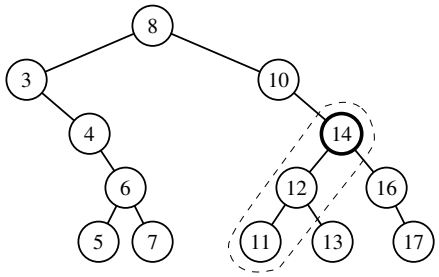


(b)

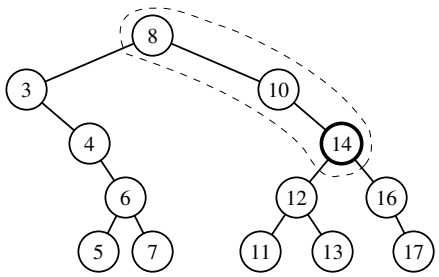


(c)

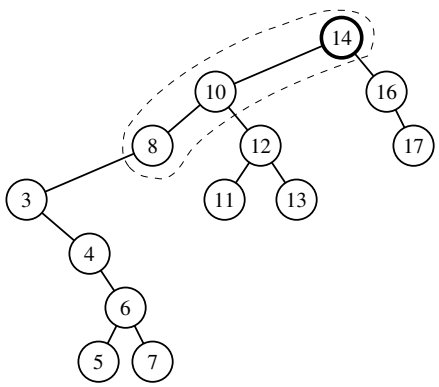
Figure 11.19: Example of splaying a node: (a) splaying the node storing 14 starts with a zig-zag; (b) after the zig-zag; (c) the next step will be a zig-zig. (Continues in Figure 11.20.)



(d)



(e)



(f)

Figure 11.20: Example of splaying a node:(d) after the zig-zig; (e) the next step is again a zig-zig; (f) after the zig-zig. (Continued from Figure 11.19.)

11.4.2 When to Splay

The rules that dictate when splaying is performed are as follows:

- When searching for key k, if k is found at position p, we splay p, else we splay the leaf position at which the search terminates unsuccessfully. For example, the splaying in Figures 11.19 and 11.20 would be performed after searching successfully for key 14 or unsuccessfully for key 15.
- When inserting key k, we splay the newly created internal node where k gets inserted. For example, the splaying in Figures 11.19 and 11.20 would be performed if 14 were the newly inserted key. We show a sequence of insertions in a splay tree in Figure 11.21.

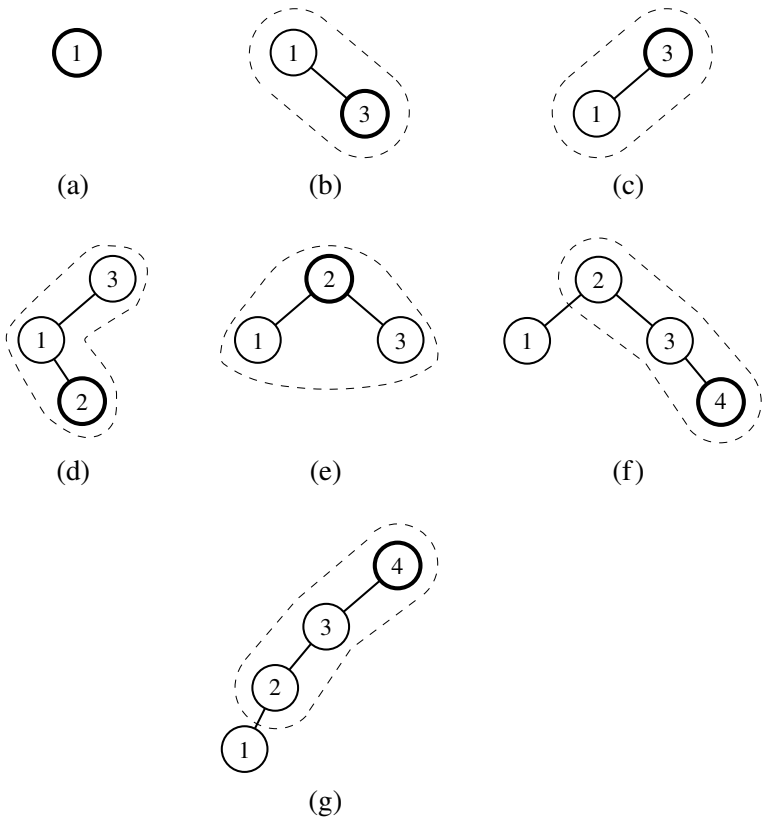


Figure 11.21: A sequence of insertions in a splay tree: (a) initial tree; (b) after inserting 3, but before a zig step; (c) after splaying; (d) after inserting 2, but before a zig-zag step; (e) after splaying; (f) after inserting 4, but before a zig-zig step; (g) after splaying.

- When deleting a key k, we splay the position p that is the parent of the removed node; recall that by the removal algorithm for binary search trees, the removed node may be that originally containing k, or a descendant node with a replacement key. An example of splaying following a deletion is shown in Figure 11.22.

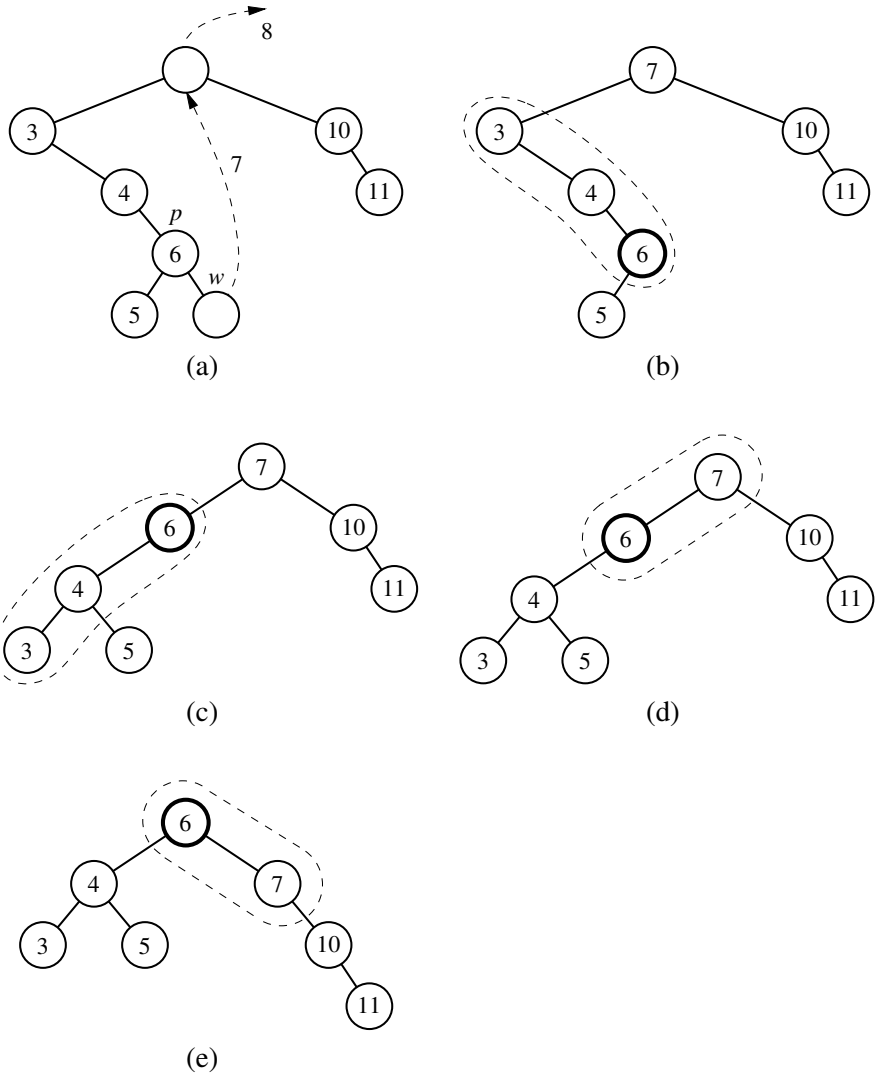


Figure 11.22: Deletion from a splay tree: (a) the deletion of 8 from the root node is performed by moving to the root the key of its inorder predecessor w, deleting w, and splaying the parent p of w; (b) splaying p starts with a zig-zig; (c) after the zig-zig; (d) the next step is a zig; (e) after the zig.

11.4.3 Python Implementation

Although the mathematical analysis of a splay tree's performance is complex (see Section 11.4.4), the *implementation* of splay trees is a rather simple adaptation to a standard binary search tree. Code Fragment 11.14 provides a complete implementation of a SplayTreeMap class, based upon the underlying TreeMap class and use of the balancing framework described in Section 11.2.1. It is important to note that our original TreeMap class makes calls to the _rebalance_access method, not just from within the __getitem__ method, but also during __setitem__ when modifying the value associated with an existing key, and after any map operations that result in a failed search.

```
1  class SplayTreeMap(TreeMap):
2    """Sorted map implementation using a splay tree."""
3    #------------------------------ splay operation ------------------------------
4    def _splay(self, p):
5      while p != self.root( ):
6        parent = self.parent(p)
7        grand = self.parent(parent)
8        if grand is None:
9          # zig case
10         self._rotate(p)
11       elif (parent == self.left(grand)) == (p == self.left(parent)):
12         # zig-zig case
13         self._rotate(parent)                    # move PARENT up
14         self._rotate(p)                         # then move p up
15       else:
16         # zig-zag case
17         self._rotate(p)                         # move p up
18         self._rotate(p)                         # move p up again
19
20   #---------------------------- override balancing hooks ----------------------------
21   def _rebalance_insert(self, p):
22     self._splay(p)
23
24   def _rebalance_delete(self, p):
25     if p is not None:
26       self._splay(p)
27
28   def _rebalance_access(self, p):
29     self._splay(p)
```

Code Fragment 11.14: A complete implementation of the SplayTreeMap class.

11.4.4 Amortized Analysis of Splaying ⋆

After a zig-zig or zig-zag, the depth of position p decreases by two, and after a zig the depth of p decreases by one. Thus, if p has depth d, splaying p consists of a sequence of $\lfloor d/2 \rfloor$ zig-zigs and/or zig-zags, plus one final zig if d is odd. Since a single zig-zig, zig-zag, or zig affects a constant number of nodes, it can be done in $O(1)$ time. Thus, splaying a position p in a binary search tree T takes time $O(d)$, where d is the depth of p in T. In other words, the time for performing a splaying step for a position p is asymptotically the same as the time needed just to reach that position in a top-down search from the root of T.

Worst-Case Time

In the worst case, the overall running time of a search, insertion, or deletion in a splay tree of height h is $O(h)$, since the position we splay might be the deepest position in the tree. Moreover, it is possible for h to be as large as n, as shown in Figure 11.21. Thus, from a worst-case point of view, a splay tree is not an attractive data structure.

In spite of its poor worst-case performance, a splay tree performs well in an amortized sense. That is, in a sequence of intermixed searches, insertions, and deletions, each operation takes on average logarithmic time. We perform the amortized analysis of splay trees using the accounting method.

Amortized Performance of Splay Trees

For our analysis, we note that the time for performing a search, insertion, or deletion is proportional to the time for the associated splaying. So let us consider only splaying time.

Let T be a splay tree with n keys, and let w be a node of T. We define the ***size*** $n(w)$ of w as the number of nodes in the subtree rooted at w. Note that this definition implies that the size of a nonleaf node is one more than the sum of the sizes of its children. We define the ***rank*** $r(w)$ of a node w as the logarithm in base 2 of the size of w, that is, $r(w) = \log(n(w))$. Clearly, the root of T has the maximum size, n, and the maximum rank, $\log n$, while each leaf has size 1 and rank 0.

We use cyber-dollars to pay for the work we perform in splaying a position p in T, and we assume that one cyber-dollar pays for a zig, while two cyber-dollars pay for a zig-zig or a zig-zag. Hence, the cost of splaying a position at depth d is d cyber-dollars. We keep a virtual account storing cyber-dollars at each position of T. Note that this account exists only for the purpose of our amortized analysis, and does not need to be included in a data structure implementing the splay tree T.

An Accounting Analysis of Splaying

When we perform a splaying, we pay a certain number of cyber-dollars (the exact value of the payment will be determined at the end of our analysis). We distinguish three cases:

- If the payment is equal to the splaying work, then we use it all to pay for the splaying.
- If the payment is greater than the splaying work, we deposit the excess in the accounts of several nodes.
- If the payment is less than the splaying work, we make withdrawals from the accounts of several nodes to cover the deficiency.

We show below that a payment of $O(\log n)$ cyber-dollars per operation is sufficient to keep the system working, that is, to ensure that each node keeps a nonnegative account balance.

An Accounting Invariant for Splaying

We use a scheme in which transfers are made between the accounts of the nodes to ensure that there will always be enough cyber-dollars to withdraw for paying for splaying work when needed.

In order to use the accounting method to perform our analysis of splaying, we maintain the following invariant:

> ***Before and after a splaying, each node*** w ***of*** T ***has*** $r(w)$ ***cyber-dollars in its account.***

Note that the invariant is "financially sound," since it does not require us to make a preliminary deposit to endow a tree with zero keys.

Let $r(T)$ be the sum of the ranks of all the nodes of T. To preserve the invariant after a splaying, we must make a payment equal to the splaying work plus the total change in $r(T)$. We refer to a single zig, zig-zig, or zig-zag operation in a splaying as a splaying ***substep***. Also, we denote the rank of a node w of T before and after a splaying substep with $r(w)$ and $r'(w)$, respectively. The following proposition gives an upper bound on the change of $r(T)$ caused by a single splaying substep. We will repeatedly use this lemma in our analysis of a full splaying of a node to the root.

Proposition 11.3: *Let δ be the variation of $r(T)$ caused by a single splaying substep (a zig, zig-zig, or zig-zag) for a node x in T. We have the following:*

- $\delta \leq 3(r'(x) - r(x)) - 2$ *if the substep is a zig-zig or zig-zag.*
- $\delta \leq 3(r'(x) - r(x))$ *if the substep is a zig.*

Justification: We use the fact (see Proposition B.1, Appendix A) that, if $a > 0$, $b > 0$, and $c > a + b$,

$$\log a + \log b < 2\log c - 2. \tag{11.6}$$

Let us consider the change in $r(T)$ caused by each type of splaying substep.

***zig-zig*:** (Recall Figure 11.16.) Since the size of each node is one more than the size of its two children, note that only the ranks of x, y, and z change in a zig-zig operation, where y is the parent of x and z is the parent of y. Also, $r'(x) = r(z)$, $r'(y) \leq r'(x)$, and $r(x) \leq r(y)$. Thus,

$$\begin{aligned} \delta &= r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(x) + r'(z) - 2r(x). \end{aligned} \tag{11.7}$$

Note that $n(x) + n'(z) < n'(x)$. Thus, $r(x) + r'(z) < 2r'(x) - 2$, as per Formula 11.6; that is,

$$r'(z) < 2r'(x) - r(x) - 2.$$

This inequality and Formula 11.7 imply

$$\begin{aligned} \delta &\leq r'(x) + (2r'(x) - r(x) - 2) - 2r(x) \\ &\leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

***zig-zag*:** (Recall Figure 11.17.) Again, by the definition of size and rank, only the ranks of x, y, and z change, where y denotes the parent of x and z denotes the parent of y. Also, $r(x) < r(y) < r(z) = r'(x)$. Thus,

$$\begin{aligned} \delta &= r'(x) + r'(y) + r'(z) - r(x) - r(y) - r(z) \\ &= r'(y) + r'(z) - r(x) - r(y) \\ &\leq r'(y) + r'(z) - 2r(x). \end{aligned} \tag{11.8}$$

Note that $n'(y) + n'(z) < n'(x)$; hence, $r'(y) + r'(z) < 2r'(x) - 2$, as per Formula 11.6. Thus,

$$\begin{aligned} \delta &\leq 2r'(x) - 2 - 2r(x) \\ &= 2(r'(x) - r(x)) - 2 \leq 3(r'(x) - r(x)) - 2. \end{aligned}$$

***zig*:** (Recall Figure 11.18.) In this case, only the ranks of x and y change, where y denotes the parent of x. Also, $r'(y) \leq r(y)$ and $r'(x) \geq r(x)$. Thus,

$$\begin{aligned} \delta &= r'(y) + r'(x) - r(y) - r(x) \\ &\leq r'(x) - r(x) \\ &\leq 3(r'(x) - r(x)). \end{aligned}$$

■

Proposition 11.4: *Let T be a splay tree with root t, and let Δ be the total variation of $r(T)$ caused by splaying a node x at depth d. We have*

$$\Delta \leq 3(r(t) - r(x)) - d + 2.$$

Justification: Splaying node x consists of $c = \lceil d/2 \rceil$ splaying substeps, each of which is a zig-zig or a zig-zag, except possibly the last one, which is a zig if d is odd. Let $r_0(x) = r(x)$ be the initial rank of x, and for $i = 1, \ldots, c$, let $r_i(x)$ be the rank of x after the ith substep and δ_i be the variation of $r(T)$ caused by the ith substep. By Proposition 11.3, the total variation Δ of $r(T)$ caused by splaying x is

$$\begin{aligned}
\Delta &= \sum_{i=1}^{c} \delta_i \\
&\leq 2 + \sum_{i=1}^{c} 3(r_i(x) - r_{i-1}(x)) - 2 \\
&= 3(r_c(x) - r_0(x)) - 2c + 2 \\
&\leq 3(r(t) - r(x)) - d + 2.
\end{aligned}$$

■

By Proposition 11.4, if we make a payment of $3(r(t) - r(x)) + 2$ cyber-dollars towards the splaying of node x, we have enough cyber-dollars to maintain the invariant, keeping $r(w)$ cyber-dollars at each node w in T, and pay for the entire splaying work, which costs d cyber-dollars. Since the size of the root t is n, its rank $r(t) = \log n$. Given that $r(x) \geq 0$, the payment to be made for splaying is $O(\log n)$ cyber-dollars. To complete our analysis, we have to compute the cost for maintaining the invariant when a node is inserted or deleted.

When inserting a new node w into a splay tree with n keys, the ranks of all the ancestors of w are increased. Namely, let $w_0, w_i, \ldots, w_d$ be the ancestors of w, where $w_0 = w$, w_i is the parent of w_{i-1}, and w_d is the root. For $i = 1, \ldots, d$, let $n'(w_i)$ and $n(w_i)$ be the size of w_i before and after the insertion, respectively, and let $r'(w_i)$ and $r(w_i)$ be the rank of w_i before and after the insertion. We have

$$n'(w_i) = n(w_i) + 1.$$

Also, since $n(w_i) + 1 \leq n(w_{i+1})$, for $i = 0, 1, \ldots, d-1$, we have the following for each i in this range:

$$r'(w_i) = \log(n'(w_i)) = \log(n(w_i) + 1) \leq \log(n(w_{i+1})) = r(w_{i+1}).$$

Thus, the total variation of $r(T)$ caused by the insertion is

$$\begin{aligned}
\sum_{i=1}^{d} \big(r'(w_i) - r(w_i)\big) &\leq r'(w_d) + \sum_{i=1}^{d-1} (r(w_{i+1}) - r(w_i)) \\
&= r'(w_d) - r(w_0) \\
&\leq \log n.
\end{aligned}$$

Therefore, a payment of $O(\log n)$ cyber-dollars is sufficient to maintain the invariant when a new node is inserted.

When deleting a node w from a splay tree with n keys, the ranks of all the ancestors of w are decreased. Thus, the total variation of $r(T)$ caused by the deletion is negative, and we do not need to make any payment to maintain the invariant when a node is deleted. Therefore, we may summarize our amortized analysis in the following proposition (which is sometimes called the "balance proposition" for splay trees):

Proposition 11.5: *Consider a sequence of m operations on a splay tree, each one a search, insertion, or deletion, starting from a splay tree with zero keys. Also, let n_i be the number of keys in the tree after operation i, and n be the total number of insertions. The total running time for performing the sequence of operations is*

$$O\left(m + \sum_{i=1}^{m} \log n_i\right),$$

which is $O(m \log n)$.

In other words, the amortized running time of performing a search, insertion, or deletion in a splay tree is $O(\log n)$, where n is the size of the splay tree at the time. Thus, a splay tree can achieve logarithmic-time amortized performance for implementing a sorted map ADT. This amortized performance matches the worst-case performance of AVL trees, $(2,4)$ trees, and red-black trees, but it does so using a simple binary tree that does not need any extra balance information stored at each of its nodes. In addition, splay trees have a number of other interesting properties that are not shared by these other balanced search trees. We explore one such additional property in the following proposition (which is sometimes called the "Static Optimality" proposition for splay trees):

Proposition 11.6: *Consider a sequence of m operations on a splay tree, each one a search, insertion, or deletion, starting from a splay tree T with zero keys. Also, let $f(i)$ denote the number of times the entry i is accessed in the splay tree, that is, its frequency, and let n denote the total number of entries. Assuming that each entry is accessed at least once, then the total running time for performing the sequence of operations is*

$$O\left(m + \sum_{i=1}^{n} f(i) \log(m/f(i))\right).$$

We omit the proof of this proposition, but it is not as hard to justify as one might imagine. The remarkable thing is that this proposition states that the amortized running time of accessing an entry i is $O(\log(m/f(i)))$.

11.5 (2,4) Trees

In this section, we consider a data structure known as a ***(2,4) tree***. It is a particular example of a more general structure known as a ***multiway search tree***, in which internal nodes may have more than two children. Other forms of multiway search trees will be discussed in Section 15.3.

11.5.1 Multiway Search Trees

Recall that general trees are defined so that internal nodes may have many children. In this section, we discuss how general trees can be used as multiway search trees. Map items stored in a search tree are pairs of the form (k, v), where k is the ***key*** and v is the ***value*** associated with the key.

Definition of a Multiway Search Tree

Let w be a node of an ordered tree. We say that w is a ***d-node*** if w has d children. We define a multiway search tree to be an ordered tree T that has the following properties, which are illustrated in Figure 11.23a:

- Each internal node of T has at least two children. That is, each internal node is a d-node such that $d \geq 2$.
- Each internal d-node w of T with children $c_1, \ldots, c_d$ stores an ordered set of $d-1$ key-value pairs $(k_1, v_1), \ldots, (k_{d-1}, v_{d-1})$, where $k_1 \leq \cdots \leq k_{d-1}$.
- Let us conventionally define $k_0 = -\infty$ and $k_d = +\infty$. For each item (k, v) stored at a node in the subtree of w rooted at c_i, $i = 1, \ldots, d$, we have that $k_{i-1} \leq k \leq k_i$.

That is, if we think of the set of keys stored at w as including the special fictitious keys $k_0 = -\infty$ and $k_d = +\infty$, then a key k stored in the subtree of T rooted at a child node c_i must be "in between" two keys stored at w. This simple viewpoint gives rise to the rule that a d-node stores $d-1$ regular keys, and it also forms the basis of the algorithm for searching in a multiway search tree.

By the above definition, the external nodes of a multiway search do not store any data and serve only as "placeholders." These external nodes can be efficiently represented by None references, as has been our convention with binary search trees (Section 11.1). However, for the sake of exposition, we will discuss these as actual nodes that do not store anything. Based on this definition, there is an interesting relationship between the number of key-value pairs and the number of external nodes in a multiway search tree.

Proposition 11.7: *An n-item multiway search tree has $n+1$ external nodes.*

We leave the justification of this proposition as an exercise (C-11.52).

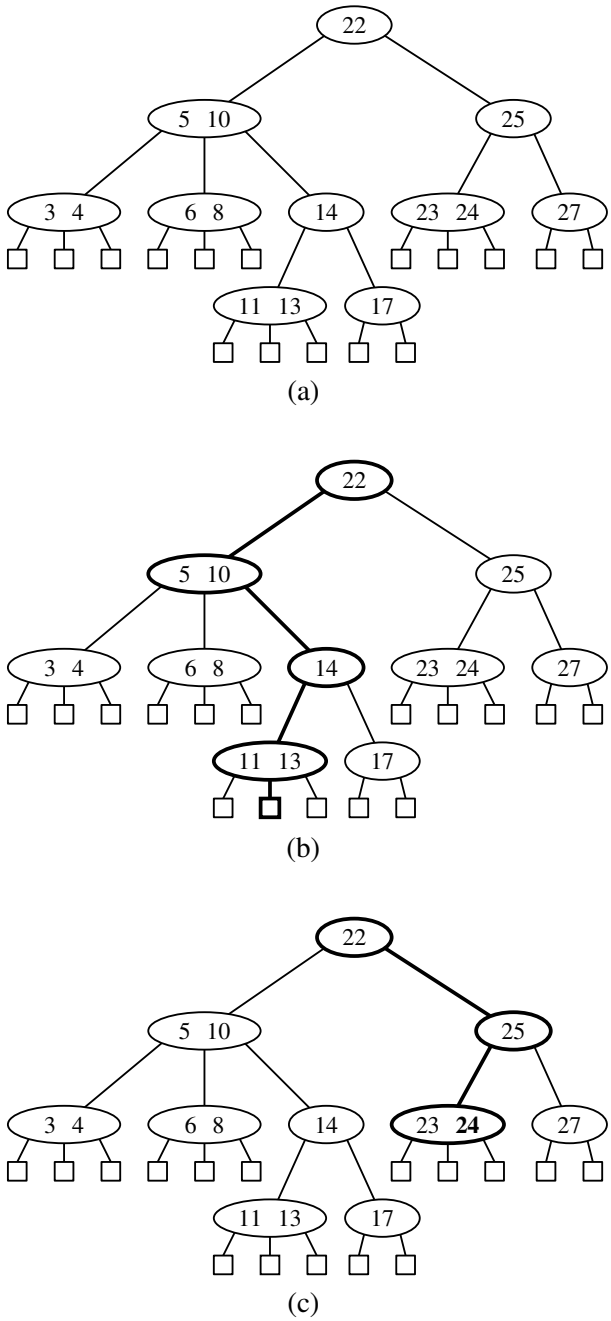


Figure 11.23: (a) A multiway search tree T; (b) search path in T for key 12 (unsuccessful search); (c) search path in T for key 24 (successful search).

Searching in a Multiway Tree

Searching for an item with key k in a multiway search tree T is simple. We perform such a search by tracing a path in T starting at the root. (See Figure 11.23b and c.) When we are at a d-node w during this search, we compare the key k with the keys $k_1, \ldots, k_{d-1}$ stored at w. If $k = k_i$ for some i, the search is successfully completed. Otherwise, we continue the search in the child c_i of w such that $k_{i-1} < k < k_i$. (Recall that we conventionally define $k_0 = -\infty$ and $k_d = +\infty$.) If we reach an external node, then we know that there is no item with key k in T, and the search terminates unsuccessfully.

Data Structures for Representing Multiway Search Trees

In Section 8.3.3, we discuss a linked data structure for representing a general tree. This representation can also be used for a multiway search tree. When using a general tree to implement a multiway search tree, we must store at each node one or more key-value pairs associated with that node. That is, we need to store with w a reference to some collection that stores the items for w.

During a search for key k in a multiway search tree, the primary operation needed when navigating a node is finding the smallest key at that node that is greater than or equal to k. For this reason, it is natural to model the information at a node itself as a sorted map, allowing use of the find_ge(k) method. We say such a map serves as a ***secondary*** data structure to support the ***primary*** data structure represented by the entire multiway search tree. This reasoning may at first seem like a circular argument, since we need a representation of a (secondary) ordered map to represent a (primary) ordered map. We can avoid any circular dependence, however, by using the ***bootstrapping*** technique, where we use a simple solution to a problem to create a new, more advanced solution.

In the context of a multiway search tree, a natural choice for the secondary structure at each node is the SortedTableMap of Section 10.3.1. Because we want to determine the associated value in case of a match for key k, and otherwise the corresponding child c_i such that $k_{i-1} < k < k_i$, we recommend having each key k_i in the secondary structure map to the pair (v_i, c_i). With such a realization of a multiway search tree T, processing a d-node w while searching for an item of T with key k can be performed using a binary search operation in $O(\log d)$ time. Let $d_{\max}$ denote the maximum number of children of any node of T, and let h denote the height of T. The search time in a multiway search tree is therefore $O(h \log d_{\max})$. If $d_{\max}$ is a constant, the running time for performing a search is $O(h)$.

The primary efficiency goal for a multiway search tree is to keep the height as small as possible. We next discuss a strategy that caps $d_{\max}$ at 4 while guaranteeing a height h that is logarithmic in n, the total number of items stored in the map.

11.5.2 (2,4)-Tree Operations

A multiway search tree that keeps the secondary data structures stored at each node small and also keeps the primary multiway tree balanced is the $(\mathbf{2},\mathbf{4})$ ***tree***, which is sometimes called a 2-4 tree or 2-3-4 tree. This data structure achieves these goals by maintaining two simple properties (see Figure 11.24):

***Size Property*:** Every internal node has at most four children.

***Depth Property*:** All the external nodes have the same depth.

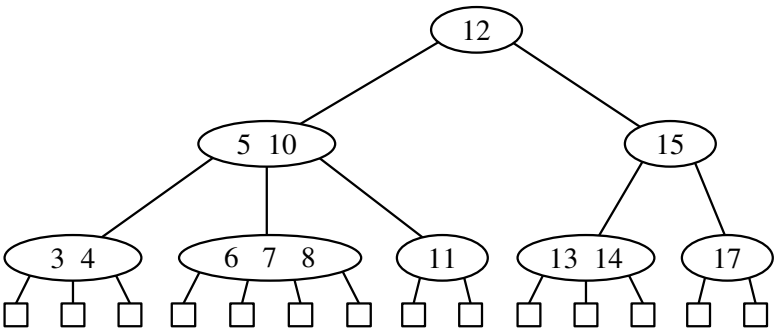


Figure 11.24: A $(2,4)$ tree.

Again, we assume that external nodes are empty and, for the sake of simplicity, we describe our search and update methods assuming that external nodes are real nodes, although this latter requirement is not strictly needed.

Enforcing the size property for $(2,4)$ trees keeps the nodes in the multiway search tree simple. It also gives rise to the alternative name "2-3-4 tree," since it implies that each internal node in the tree has 2, 3, or 4 children. Another implication of this rule is that we can represent the secondary map stored at each internal node using an unordered list or an ordered array, and still achieve $O(1)$-time performance for all operations (since $d_{\max} = 4$). The depth property, on the other hand, enforces an important bound on the height of a $(2,4)$ tree.

Proposition 11.8: *The height of a $(2,4)$ tree storing n items is $O(\log n)$.*

Justification: Let h be the height of a $(2,4)$ tree T storing n items. We justify the proposition by showing the claim

$$\frac{1}{2}\log(n+1) \leq h \leq \log(n+1). \tag{11.9}$$

To justify this claim note first that, by the size property, we can have at most 4 nodes at depth 1, at most 4^2 nodes at depth 2, and so on. Thus, the number of external nodes in T is at most 4^h. Likewise, by the depth property and the definition

of a $(2,4)$ tree, we must have at least 2 nodes at depth 1, at least 2^2 nodes at depth 2, and so on. Thus, the number of external nodes in T is at least 2^h. In addition, by Proposition 11.7, the number of external nodes in T is $n+1$. Therefore, we obtain

$$2^h \leq n+1 \leq 4^h.$$

Taking the logarithm in base 2 of the terms for the above inequalities, we get that

$$h \leq \log(n+1) \leq 2h,$$

which justifies our claim (Formula 11.9) when terms are rearranged. ■

Proposition 11.8 states that the size and depth properties are sufficient for keeping a multiway tree balanced. Moreover, this proposition implies that performing a search in a $(2,4)$ tree takes $O(\log n)$ time and that the specific realization of the secondary structures at the nodes is not a crucial design choice, since the maximum number of children $d_{\max}$ is a constant.

Maintaining the size and depth properties requires some effort after performing insertions and deletions in a $(2,4)$ tree, however. We discuss these operations next.

Insertion

To insert a new item (k,v), with key k, into a $(2,4)$ tree T, we first perform a search for k. Assuming that T has no item with key k, this search terminates unsuccessfully at an external node z. Let w be the parent of z. We insert the new item into node w and add a new child y (an external node) to w on the left of z.

Our insertion method preserves the depth property, since we add a new external node at the same level as existing external nodes. Nevertheless, it may violate the size property. Indeed, if a node w was previously a 4-node, then it would become a 5-node after the insertion, which causes the tree T to no longer be a $(2,4)$ tree. This type of violation of the size property is called an ***overflow*** at node w, and it must be resolved in order to restore the properties of a $(2,4)$ tree. Let $c_1,\ldots,c_5$ be the children of w, and let $k_1,\ldots,k_4$ be the keys stored at w. To remedy the overflow at node w, we perform a ***split*** operation on w as follows (see Figure 11.25):

- Replace w with two nodes w' and w'', where
  - w' is a 3-node with children c_1,c_2,c_3 storing keys k_1 and k_2
  - w'' is a 2-node with children c_4,c_5 storing key k_4.
- If w is the root of T, create a new root node u; else, let u be the parent of w.
- Insert key k_3 into u and make w' and w'' children of u, so that if w was child i of u, then w' and w'' become children i and $i+1$ of u, respectively.

As a consequence of a split operation on node w, a new overflow may occur at the parent u of w. If such an overflow occurs, it triggers in turn a split at node u. (See Figure 11.26.) A split operation either eliminates the overflow or propagates it into the parent of the current node. We show a sequence of insertions in a $(2,4)$ tree in Figure 11.27.

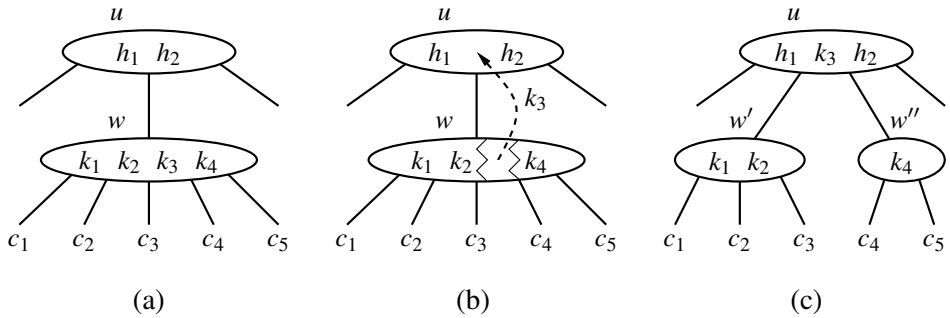


Figure 11.25: A node split: (a) overflow at a 5-node w; (b) the third key of w inserted into the parent u of w; (c) node w replaced with a 3-node w' and a 2-node w''.

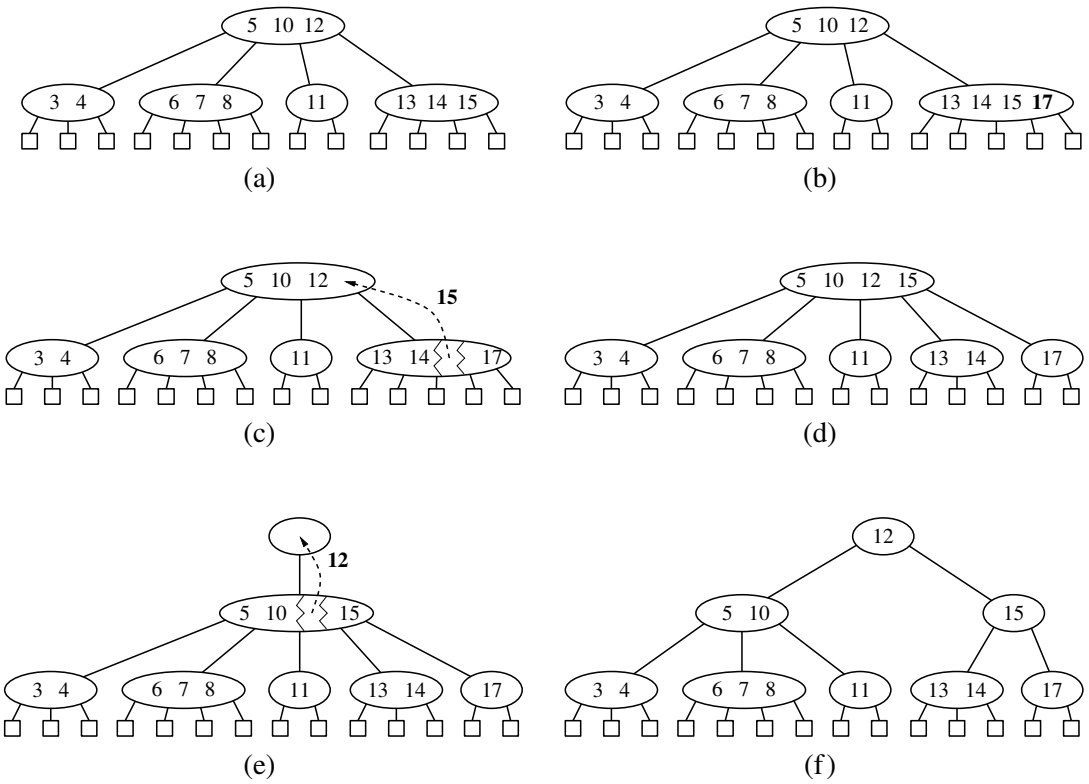


Figure 11.26: An insertion in a $(2,4)$ tree that causes a cascading split: (a) before the insertion; (b) insertion of 17, causing an overflow; (c) a split; (d) after the split a new overflow occurs; (e) another split, creating a new root node; (f) final tree.

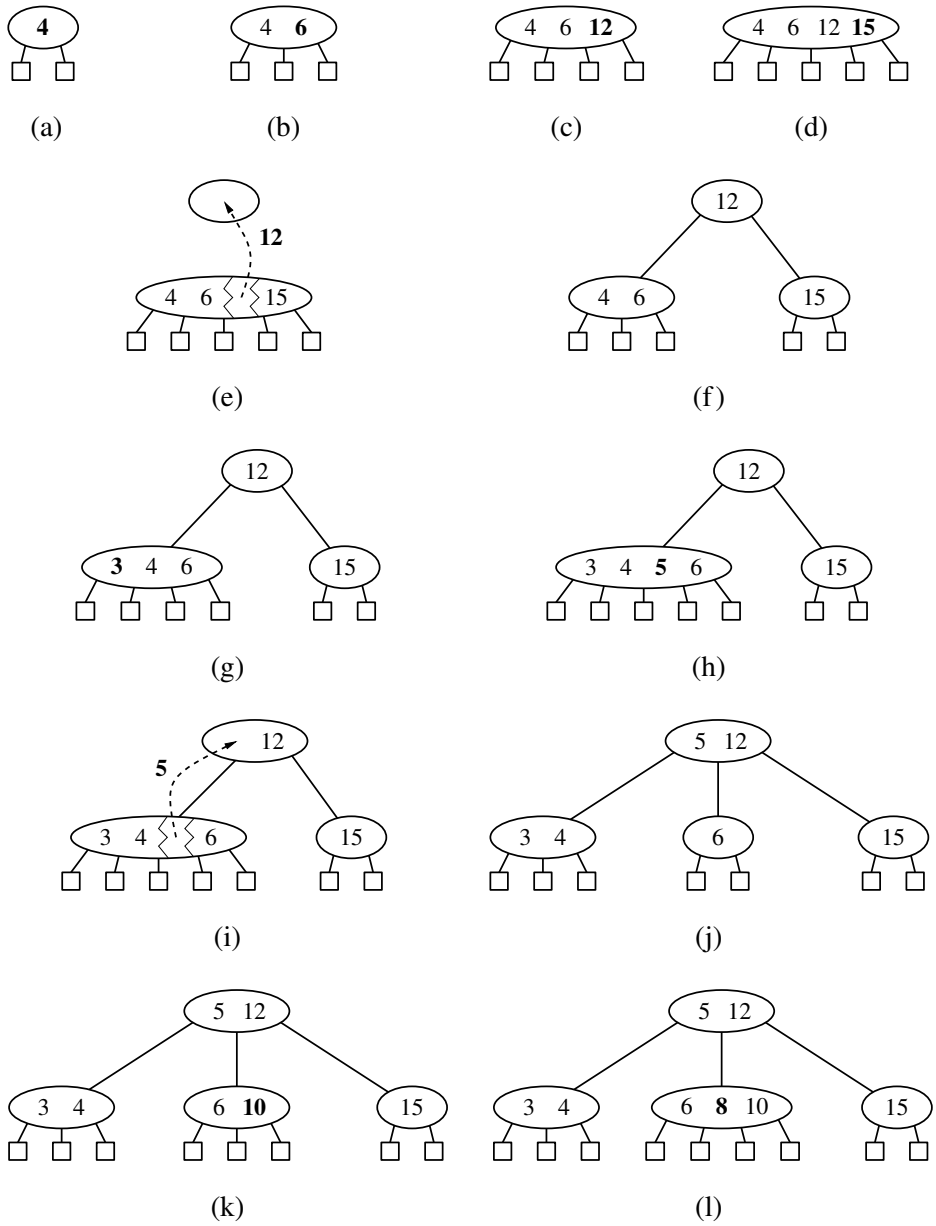


Figure 11.27: A sequence of insertions into a $(2,4)$ tree: (a) initial tree with one item; (b) insertion of 6; (c) insertion of 12; (d) insertion of 15, which causes an overflow; (e) split, which causes the creation of a new root node; (f) after the split; (g) insertion of 3; (h) insertion of 5, which causes an overflow; (i) split; (j) after the split; (k) insertion of 10; (l) insertion of 8.

Analysis of Insertion in a (2,4) Tree

Because $d_{\max}$ is at most 4, the original search for the placement of new key k uses $O(1)$ time at each level, and thus $O(\log n)$ time overall, since the height of the tree is $O(\log n)$ by Proposition 11.8.

The modifications to a single node to insert a new key and child can be implemented to run in $O(1)$ time, as can a single split operation. The number of cascading split operations is bounded by the height of the tree, and so that phase of the insertion process also runs in $O(\log n)$ time. Therefore, the total time to perform an insertion in a $(2,4)$ tree is $O(\log n)$.

Deletion

Let us now consider the removal of an item with key k from a $(2,4)$ tree T. We begin such an operation by performing a search in T for an item with key k. Removing an item from a $(2,4)$ tree can always be reduced to the case where the item to be removed is stored at a node w whose children are external nodes. Suppose, for instance, that the item with key k that we wish to remove is stored in the ith item (k_i, v_i) at a node z that has only internal-node children. In this case, we swap the item (k_i, v_i) with an appropriate item that is stored at a node w with external-node children as follows (see Figure 11.28d):

1. We find the rightmost internal node w in the subtree rooted at the ith child of z, noting that the children of node w are all external nodes.
2. We swap the item (k_i, v_i) at z with the last item of w.

Once we ensure that the item to remove is stored at a node w with only external-node children (because either it was already at w or we swapped it into w), we simply remove the item from w and remove the ith external node of w.

Removing an item (and a child) from a node w as described above preserves the depth property, for we always remove an external child from a node w with only external children. However, in removing such an external node, we may violate the size property at w. Indeed, if w was previously a 2-node, then it becomes a 1-node with no items after the removal (Figure 11.28a and d), which is not allowed in a $(2,4)$ tree. This type of violation of the size property is called an ***underflow*** at node w. To remedy an underflow, we check whether an immediate sibling of w is a 3-node or a 4-node. If we find such a sibling s, then we perform a ***transfer*** operation, in which we move a child of s to w, a key of s to the parent u of w and s, and a key of u to w. (See Figure 11.28b and c.) If w has only one sibling, or if both immediate siblings of w are 2-nodes, then we perform a ***fusion*** operation, in which we merge w with a sibling, creating a new node w', and move a key from the parent u of w to w'. (See Figure 11.28e and f.)

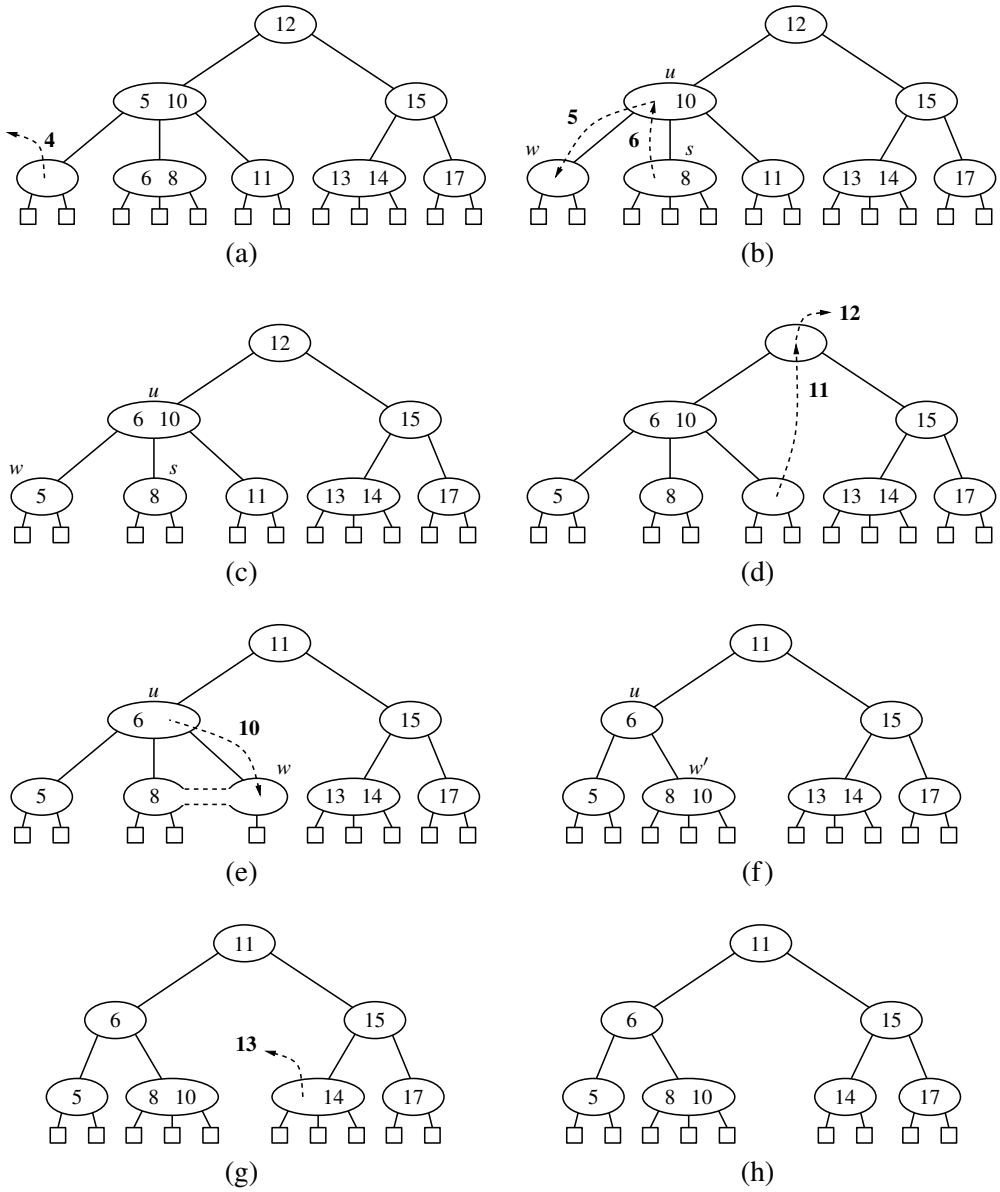


Figure 11.28: A sequence of removals from a $(2,4)$ tree: (a) removal of 4, causing an underflow; (b) a transfer operation; (c) after the transfer operation; (d) removal of 12, causing an underflow; (e) a fusion operation; (f) after the fusion operation; (g) removal of 13; (h) after removing 13.

A fusion operation at node w may cause a new underflow to occur at the parent u of w, which in turn triggers a transfer or fusion at u. (See Figure 11.29.) Hence, the number of fusion operations is bounded by the height of the tree, which is $O(\log n)$ by Proposition 11.8. If an underflow propagates all the way up to the root, then the root is simply deleted. (See Figure 11.29c and d.)

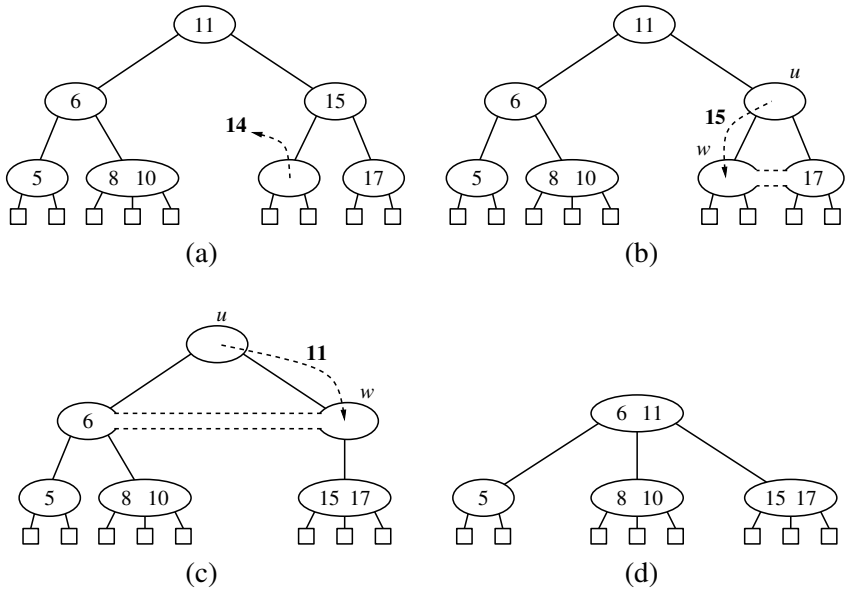


Figure 11.29: A propagating sequence of fusions in a $(2,4)$ tree: (a) removal of 14, which causes an underflow; (b) fusion, which causes another underflow; (c) second fusion operation, which causes the root to be removed; (d) final tree.

Performance of (2,4) Trees

The asymptotic performance of a $(2,4)$ tree is identical to that of an AVL tree (see Table 11.2) in terms of the sorted map ADT, with guaranteed logarithmic bounds for most operations. The time complexity analysis for a $(2,4)$ tree having n key-value pairs is based on the following:

- The height of a $(2,4)$ tree storing n entries is $O(\log n)$, by Proposition 11.8.
- A split, transfer, or fusion operation takes $O(1)$ time.
- A search, insertion, or removal of an entry visits $O(\log n)$ nodes.

Thus, $(2,4)$ trees provide for fast map search and update operations. $(2,4)$ trees also have an interesting relationship to the data structure we discuss next.

11.6 Red-Black Trees

Although AVL trees and $(2,4)$ trees have a number of nice properties, they also have some disadvantages. For instance, AVL trees may require many restructure operations (rotations) to be performed after a deletion, and $(2,4)$ trees may require many split or fusing operations to be performed after an insertion or removal. The data structure we discuss in this section, the red-black tree, does not have these drawbacks; it uses $O(1)$ structural changes after an update in order to stay balanced.

Formally, a ***red-black tree*** is a binary search tree (see Section 11.1) with nodes colored red and black in a way that satisfies the following properties:

***Root Property*:** The root is black.

***Red Property*:** The children of a red node (if any) are black.

Depth Property*:** All nodes with zero or one children have the same ***black depth, defined as the number of black ancestors. (Recall that a node is its own ancestor).

An example of a red-black tree is shown in Figure 11.30.

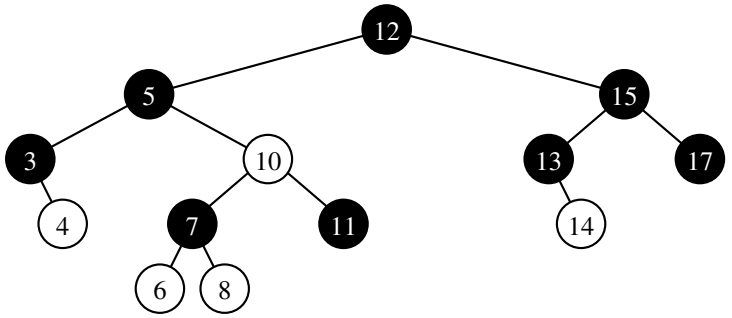


Figure 11.30: An example of a red-black tree, with "red" nodes drawn in white. The common black depth for this tree is 3.

We can make the red-black tree definition more intuitive by noting an interesting correspondence between red-black trees and $(2,4)$ trees (excluding their trivial external nodes). Namely, given a red-black tree, we can construct a corresponding $(2,4)$ tree by merging every red node w into its parent, storing the entry from w at its parent, and with the children of w becoming ordered children of the parent. For example, the red-black tree in Figure 11.30 corresponds to the $(2,4)$ tree from Figure 11.24, as illustrated in Figure 11.31. The depth property of the red-black tree corresponds to the depth property of the $(2,4)$ tree since exactly one black node of the red-black tree contributes to each node of the corresponding $(2,4)$ tree.

Conversely, we can transform any $(2,4)$ tree into a corresponding red-black tree by coloring each node w black and then performing the following transformations, as illustrated in Figure 11.32.

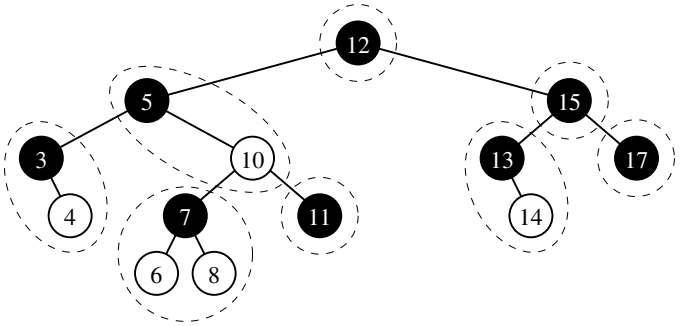


Figure 11.31: An illustration that the red-black tree of Figure 11.30 corresponds to the $(2,4)$ tree of Figure 11.24, based on the highlighted grouping of red nodes with their black parents.

- If w is a 2-node, then keep the (black) children of w as is.
- If w is a 3-node, then create a new red node y, give w's last two (black) children to y, and make the first child of w and y be the two children of w.
- If w is a 4-node, then create two new red nodes y and z, give w's first two (black) children to y, give w's last two (black) children to z, and make y and z be the two children of w.

Notice that a red node always has a black parent in this construction.

Proposition 11.9: *The height of a red-black tree storing n entries is $O(\log n)$.*

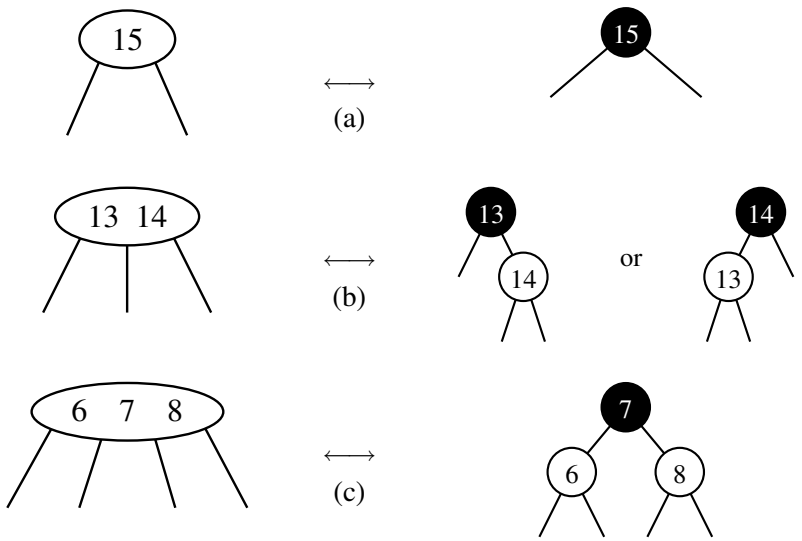


Figure 11.32: Correspondence between nodes of a $(2,4)$ tree and a red-black tree: (a) 2-node; (b) 3-node; (c) 4-node.

Justification: Let T be a red-black tree storing n entries, and let h be the height of T. We justify this proposition by establishing the following fact:

$$\log(n+1) - 1 \leq h \leq 2\log(n+1) - 2.$$

Let d be the common black depth of all nodes of T having zero or one children. Let T' be the $(2,4)$ tree associated with T, and let h' be the height of T' (excluding trivial leaves). Because of the correspondence between red-black trees and $(2,4)$ trees, we know that $h' = d$. Hence, by Proposition 11.8, $d = h' \leq \log(n+1) - 1$. By the red property, $h \leq 2d$. Thus, we obtain $h \leq 2\log(n+1) - 2$. The other inequality, $\log(n+1) - 1 \leq h$, follows from Proposition 8.8 and the fact that T has n nodes. ■

11.6.1 Red-Black Tree Operations

The algorithm for searching in a red-black tree T is the same as that for a standard binary search tree (Section 11.1). Thus, searching in a red-black tree takes time proportional to the height of the tree, which is $O(\log n)$ by Proposition 11.9.

The correspondence between $(2,4)$ trees and red-black trees provides important intuition that we will use in our discussion of how to perform updates in red-black trees; in fact, the update algorithms for red-black trees can seem mysteriously complex without this intuition. Split and fuse operations of a $(2,4)$ tree will be effectively mimicked by recoloring neighboring red-black tree nodes. A rotation within a red-black tree will be used to change orientations of a 3-node between the two forms shown in Figure 11.32(b).

Insertion

Now consider the insertion of a key-value pair (k,v) into a red-black tree T. The algorithm initially proceeds as in a standard binary search tree (Section 11.1.3). Namely, we search for k in T until we reach a null subtree, and we introduce a new leaf x at that position, storing the item. In the special case that x is the only node of T, and thus the root, we color it black. In all other cases, we color x red. This action corresponds to inserting (k,v) into a node of the $(2,4)$ tree T' with external children. The insertion preserves the root and depth properties of T, but it may violate the red property. Indeed, if x is not the root of T and the parent y of x is red, then we have a parent and a child (namely, y and x) that are both red. Note that by the root property, y cannot be the root of T, and by the red property (which was previously satisfied), the parent z of y must be black. Since x and its parent are red, but x's grandparent z is black, we call this violation of the red property a ***double red*** at node x. To remedy a double red, we consider two cases.

Case 1: ***The Sibling s of y Is Black (or None).*** (See Figure 11.33.) In this case, the double red denotes the fact that we have added the new node to a corresponding 3-node of the $(2,4)$ tree T', effectively creating a malformed 4-node. This formation has one red node (y) that is the parent of another red node (x), while we want it to have the two red nodes as siblings instead. To fix this problem, we perform a ***trinode restructuring*** of T. The trinode restructuring is done by the operation $\mathsf{restructure}(x)$, which consists of the following steps (see again Figure 11.33; this operation is also discussed in Section 11.2):

- Take node x, its parent y, and grandparent z, and temporarily relabel them as a, b, and c, in left-to-right order, so that a, b, and c will be visited in this order by an inorder tree traversal.
- Replace the grandparent z with the node labeled b, and make nodes a and c the children of b, keeping inorder relationships unchanged.

After performing the $\mathsf{restructure}(x)$ operation, we color b black and we color a and c red. Thus, the restructuring eliminates the double-red problem. Notice that the portion of any path through the restructured part of the tree is incident to exactly one black node, both before and after the trinode restructuring. Therefore, the black depth of the tree is unaffected.

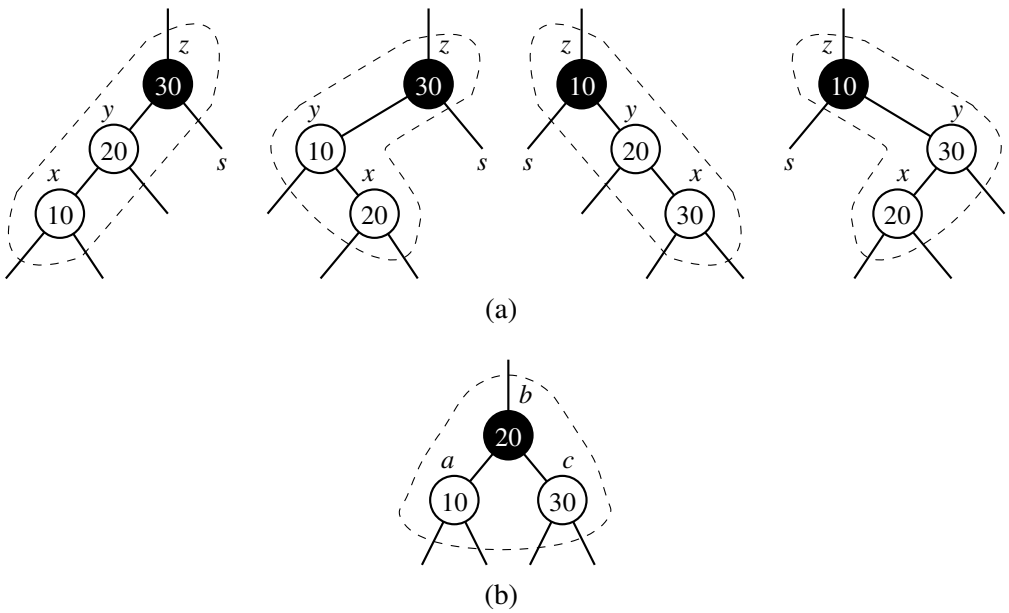


Figure 11.33: Restructuring a red-black tree to remedy a double red: (a) the four configurations for x, y, and z before restructuring; (b) after restructuring.

Case 2: ***The Sibling s of y Is Red.*** (See Figure 11.34.) In this case, the double red denotes an overflow in the corresponding $(2,4)$ tree T'. To fix the problem, we perform the equivalent of a split operation. Namely, we do a ***recoloring***: we color y and s black and their parent z red (unless z is the root, in which case, it remains black). Notice that unless z is the root, the portion of any path through the affected part of the tree is incident to exactly one black node, both before and after the recoloring. Therefore, the black depth of the tree is unaffected by the recoloring unless z is the root, in which case it is increased by one.

However, it is possible that the double-red problem reappears after such a recoloring, albeit higher up in the tree T, since z may have a red parent. If the double-red problem reappears at z, then we repeat the consideration of the two cases at z. Thus, a recoloring either eliminates the double-red problem at node x, or propagates it to the grandparent z of x. We continue going up T performing recolorings until we finally resolve the double-red problem (with either a final recoloring or a trinode restructuring). Thus, the number of recolorings caused by an insertion is no more than half the height of tree T, that is, $O(\log n)$ by Proposition 11.9.

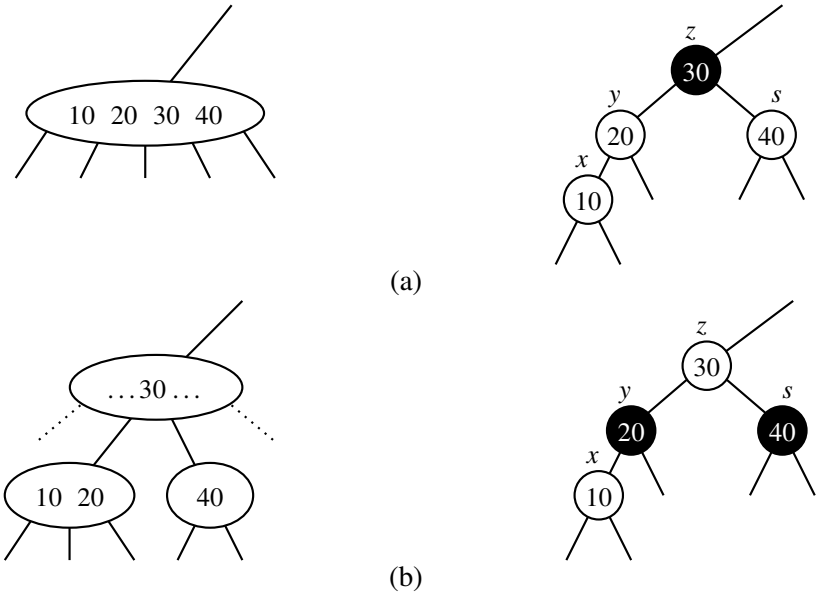


Figure 11.34: Recoloring to remedy the double-red problem: (a) before recoloring and the corresponding 5-node in the associated $(2,4)$ tree before the split; (b) after recoloring and the corresponding nodes in the associated $(2,4)$ tree after the split.

As further examples, Figures 11.35 and 11.36 show a sequence of insertion operations in a red-black tree.

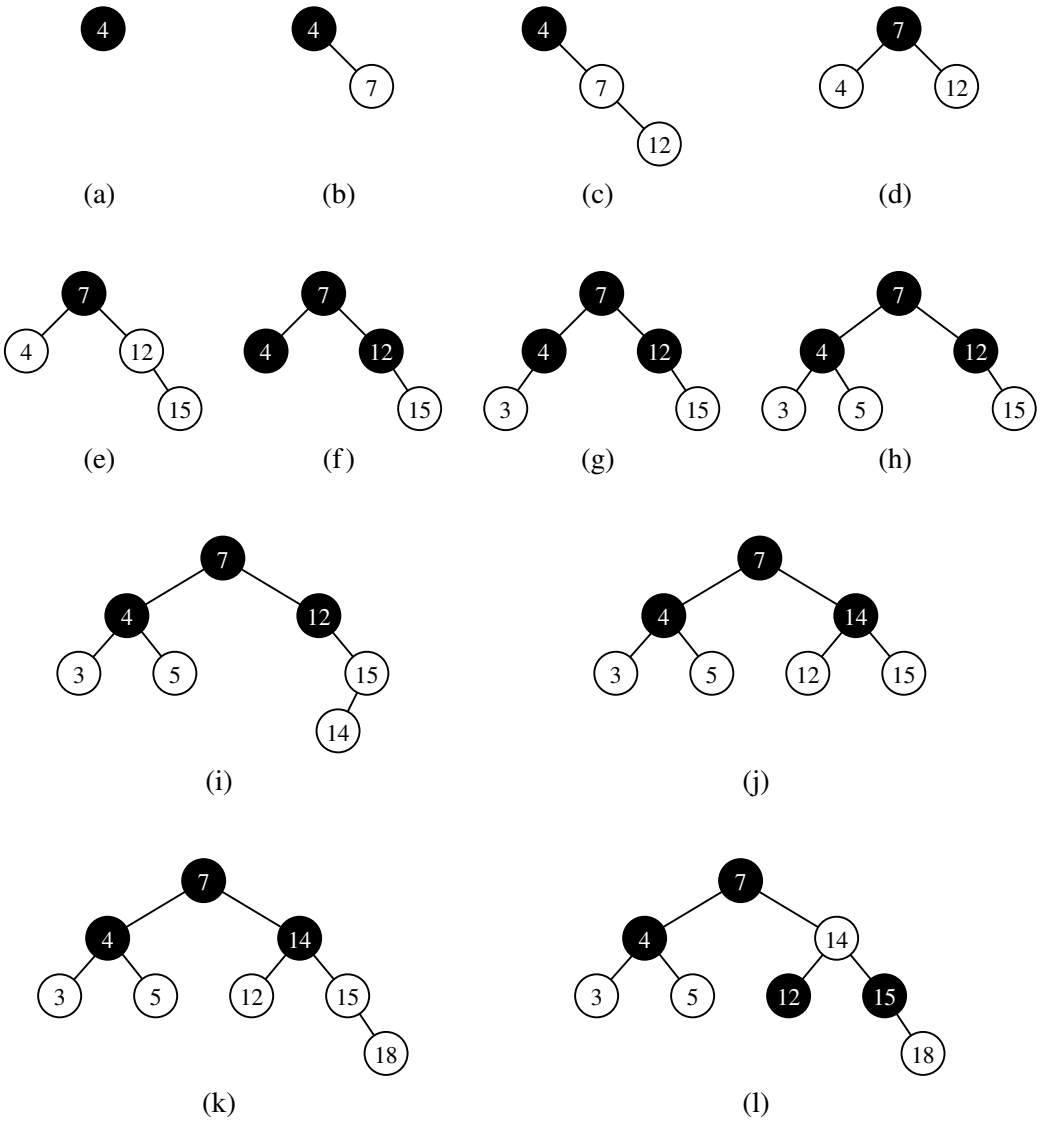


Figure 11.35: A sequence of insertions in a red-black tree: (a) initial tree; (b) insertion of 7; (c) insertion of 12, which causes a double red; (d) after restructuring; (e) insertion of 15, which causes a double red; (f) after recoloring (the root remains black); (g) insertion of 3; (h) insertion of 5; (i) insertion of 14, which causes a double red; (j) after restructuring; (k) insertion of 18, which causes a double red; (l) after recoloring. (Continues in Figure 11.36.)

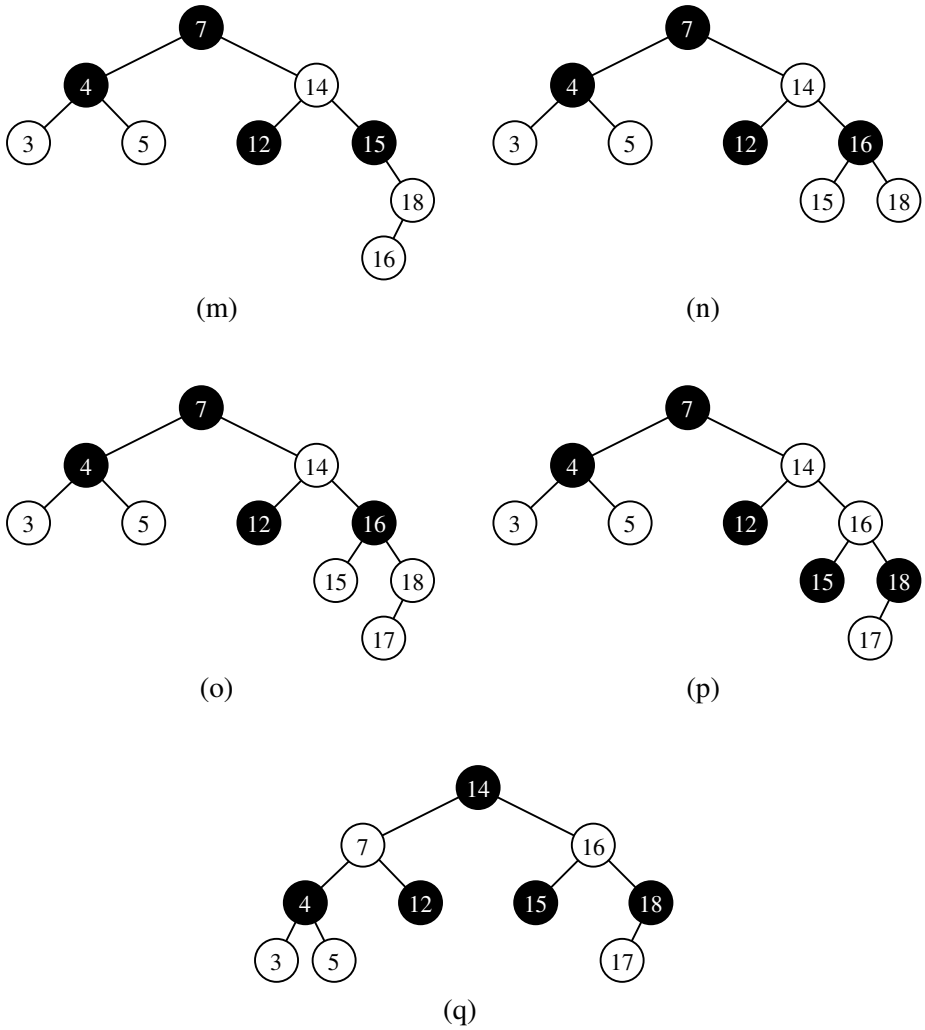


Figure 11.36: A sequence of insertions in a red-black tree: (m) insertion of 16, which causes a double red; (n) after restructuring; (o) insertion of 17, which causes a double red; (p) after recoloring there is again a double red, to be handled by a restructuring; (q) after restructuring. (Continued from Figure 11.35.)

Deletion

Deleting an item with key k from a red-black tree T initially proceeds as for a binary search tree (Section 11.1.3). Structurally, the process results in the removal a node that has at most one child (either that originally containing key k or its inorder predecessor) and the promotion of its remaining child (if any).

If the removed node was red, this structural change does not affect the black depths of any paths in the tree, nor introduce any red violations, and so the resulting tree remains a valid red-black tree. In the corresponding $(2,4)$ tree T', this case denotes the shrinking of a 3-node or 4-node. If the removed node was black, then it either had zero children or it had one child that was a red leaf (because the null subtree of the removed node has black height 0). In the latter case, the removed node represents the black part of a corresponding 3-node, and we restore the red-black properties by recoloring the promoted child to black.

The more complex case is when a (nonroot) black leaf is removed. In the corresponding $(2,4)$ tree, this denotes the removal of an item from a 2-node. Without rebalancing, such a change results in a deficit of one for the black depth along the path leading to the deleted item. By necessity, the removed node must have a sibling whose subtree has black height 1 (given that this was a valid red-black tree prior to the deletion of the black leaf).

To remedy this scenario, we consider a more general setting with a node z that is known to have two subtrees, T_{heavy} and T_{light}, such that the root of T_{light} (if any) is black and such that the black depth of T_{heavy} is exactly one more than that of T_{light}, as portrayed in Figure 11.37. In the case of a removed black leaf, z is the parent of that leaf and T_{light} is trivially the empty subtree that remains after the deletion. We describe the more general case of a deficit because our algorithm for rebalancing the tree will, in some cases, push the deficit higher in the tree (just as the resolution of a deletion in a $(2,4)$ tree sometimes cascades upward). We let y denote the root of T_{heavy}. (Such a node exists because T_{heavy} has black height at least one.)

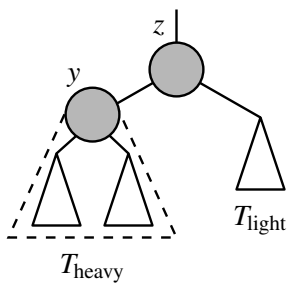


Figure 11.37: Portrayal of a deficit between the black heights of subtrees of node z. The gray color in illustrating y and z denotes the fact that these nodes may be colored either black or red.

We consider three possible cases to remedy a deficit.

Case 1: ***Node y Is Black and Has a Red Child x***. (See Figure 11.38.)

We perform a ***trinode restructuring***, as originally described in Section 11.2. The operation restructure(x) takes the node x, its parent y, and grandparent z, labels them temporarily left to right as a, b, and c, and replaces z with the node labeled b, making it the parent of the other two. We color a and c black, and give b the former color of z.

Notice that the path to T_{light} in the result includes one additional black node after the restructure, thereby resolving its deficit. In contrast, the number of black nodes on paths to any of the other three subtrees illustrated in Figure 11.38 remains unchanged.

Resolving this case corresponds to a transfer operation in the $(2,4)$ tree T' between the two children of the node with z. The fact that y has a red child assures us that it represents either a 3-node or a 4-node. In effect, the item previously stored at z is demoted to become a new 2-node to resolve the deficiency, while an item stored at y or its child is promoted to take the place of the item previously stored at z.

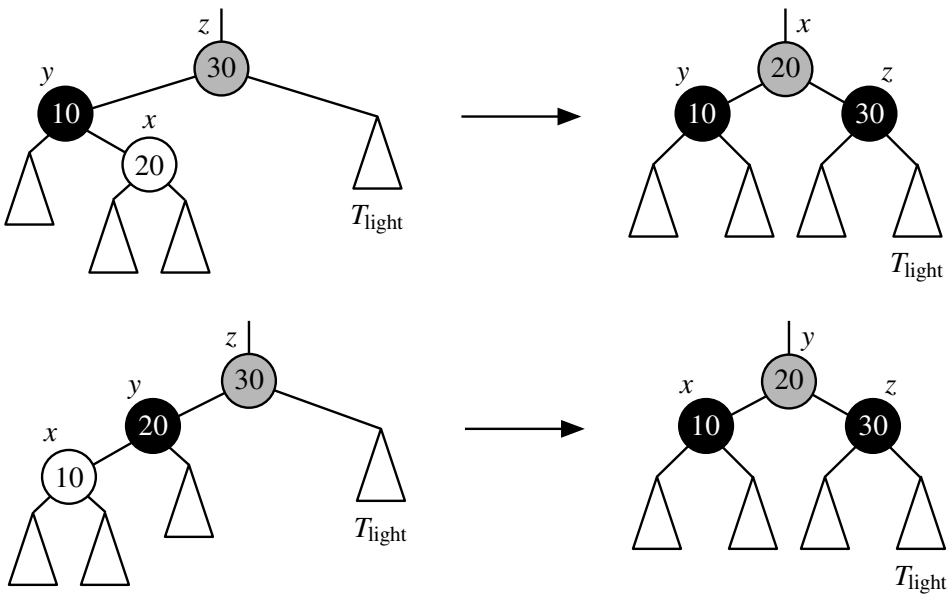


Figure 11.38: Resolving a black deficit in T_{light} by performing a trinode restructuring as restructure(x). Two possible configurations are shown (two other configurations are symmetric). The gray color of z in the left figures denotes the fact that this node may be colored either red or black. The root of the restructured portion is given that same color, while the children of that node are both colored black in the result.

Case 2: ***Node y Is Black and Both Children of y Are Black (or None).***

Resolving this case corresponds to a fusion operation in the corresponding $(2,4)$ tree T', as y must represent a 2-node. We do a ***recoloring***; we color y red, and, if z is red, we color it black. (See Figure 11.39). This does not introduce any red violation, because y does not have a red child.

In the case that z was originally red, and thus the parent in the corresponding $(2,4)$ tree is a 3-node or 4-node, this recoloring resolves the deficit. (See Figure 11.39a.) The path leading to T_{light} includes one additional black node in the result, while the recoloring did not affect the number of black nodes on the path to the subtrees of T_{heavy}.

In the case that z was originally black, and thus the parent in the corresponding $(2,4)$ tree is a 2-node, the recoloring has not increased the number of black nodes on the path to T_{light}; in fact, it has *reduced* the number of black nodes on the path to T_{heavy}. (See Figure 11.39b.) After this step, the two children of z will have the same black height. However, the entire tree rooted at z has become deficient, thereby propogating the problem higher in the tree; we must repeat consideration of all three cases at the parent of z as a remedy.

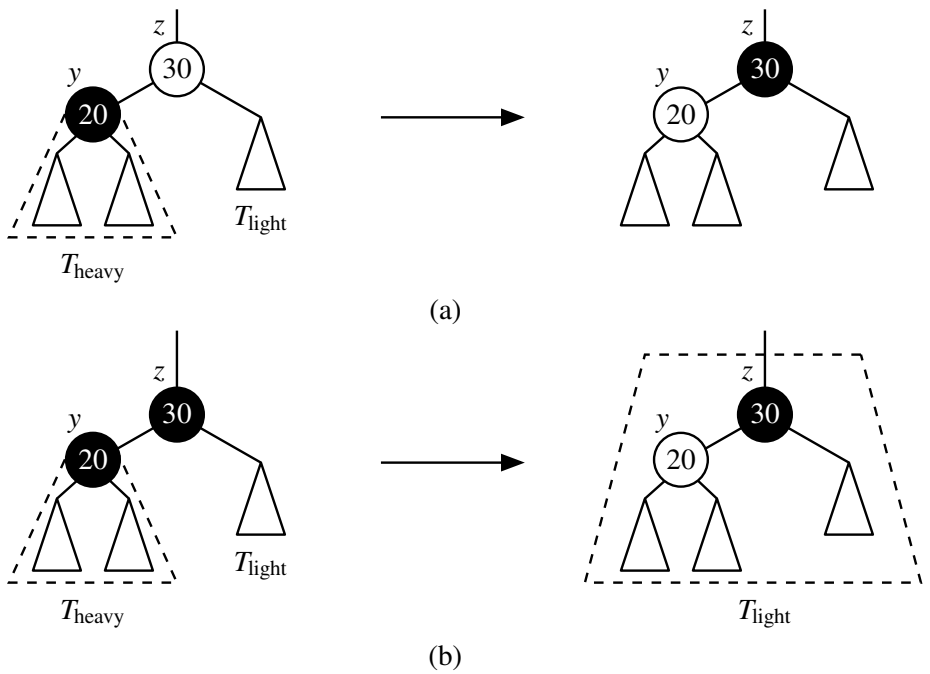


Figure 11.39: Resolving a black deficit in T_{light} by a recoloring operation: (a) when z is originally red, reversing the colors of y and z resolves the black deficit in T_{light}, ending the process; (b) when z is originally black, recoloring y causes the entire subtree of z to have a black deficit, requiring a cascading remedy.

Case 3: ***Node y Is Red***. (See Figure 11.40.)

Because y is red and T_{heavy} has black depth at least 1, z must be black and the two subtrees of y must each have a black root and a black depth equal to that of T_{heavy}. In this case, we perform a rotation about y and z, and then recolor y black and z red. This denotes a reorientation of a 3-node in the corresponding $(2,4)$ tree T'.

This does not immediately resolve the deficit, as the new subtree of z is an old subtree of y with black root y' and black height equal to that of the original T_{heavy}. We reapply the algorithm to resolve the deficit at z, knowing that the new child y', that is the root of T_{heavy} is now black, and therefore that either Case 1 applies or Case 2 applies. Furthermore, the next application will be the last, because Case 1 is always terminal and Case 2 will be terminal given that z is red.

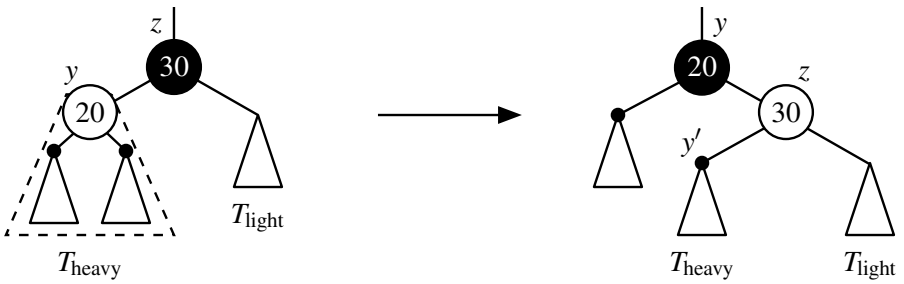


Figure 11.40: A rotation and recoloring about red node y and black node z, assuming a black deficit at z. This amounts to a change of orientation in the corresponding 3-node of a $(2,4)$ tree. This operation does not affect the black depth of any paths through this portion of the tree. Furthermore, because y was originally red, the new subtree of z must have a black root y' and must have black height equal to the original T_{heavy}. Therefore, a black deficit remains at node z after the transformation.

In Figure 11.41, we show a sequence of deletions on a red-black tree. A dashed edge in those figures, such as to the right of 7 in part (c), represents a branch with a black deficiency that has not yet been resolved. We illustrate a Case 1 restructuring in parts (c) and (d). We illustrate a Case 2 recoloring in parts (f) and (g). Finally, we show an example of a Case 3 rotation between parts (i) and (j), concluding with a Case 2 recoloring in part (k).

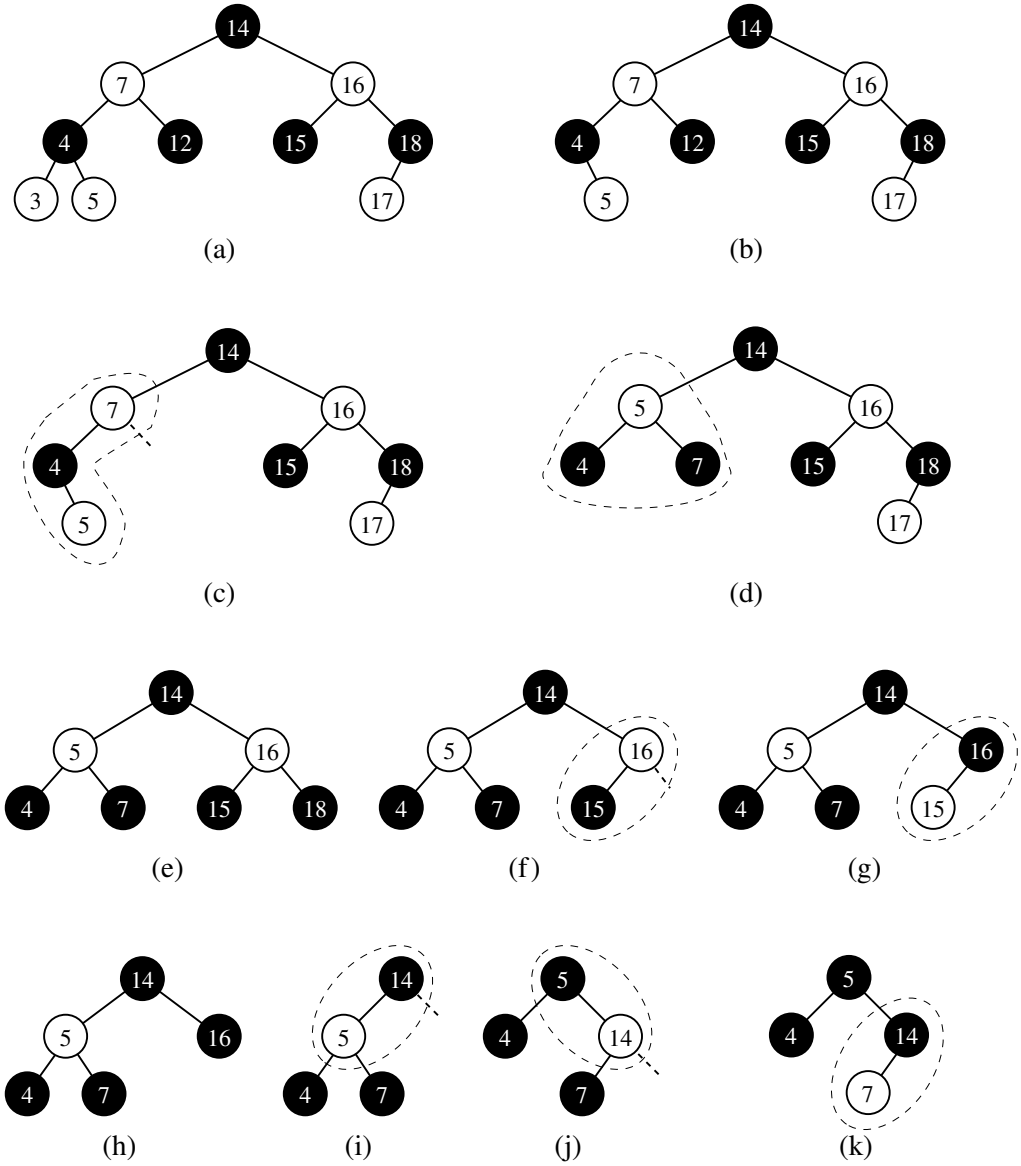


Figure 11.41: A sequence of deletions from a red-black tree: (a) initial tree; (b) removal of 3; (c) removal of 12, causing a black deficit to the right of 7 (handled by restructuring); (d) after restructuring; (e) removal of 17; (f) removal of 18, causing a black deficit to the right of 16 (handled by recoloring); (g) after recoloring; (h) removal of 15; (i) removal of 16, causing a black deficit to the right of 14 (handled initially by a rotation); (j) after the rotation the black deficit needs to be handled by a recoloring; (k) after the recoloring.

Performance of Red-Black Trees

The asymptotic performance of a red-black tree is identical to that of an AVL tree or a $(2,4)$ tree in terms of the sorted map ADT, with guaranteed logarithmic time bounds for most operations. (See Table 11.2 for a summary of the AVL performance.) The primary advantage of a red-black tree is that an insertion or deletion requires only a ***constant number of restructuring operations***. (This is in contrast to AVL trees and $(2,4)$ trees, both of which require a logarithmic number of structural changes per map operation in the worst case.) That is, an insertion or deletion in a red-black tree requires logarithmic time for a search, and may require a logarithmic number of recoloring operations that cascade upward. Yet we show, in the following propositions, that there are a constant number of rotations or restructure operations for a single map operation.

Proposition 11.10: *The insertion of an item in a red-black tree storing n items can be done in $O(\log n)$ time and requires $O(\log n)$ recolorings and at most one trinode restructuring.*

Justification: Recall that an insertion begins with a downward search, the creation of a new leaf node, and then a potential upward effort to remedy a double-red violation. There may be logarithmically many recoloring operations due to an upward cascading of Case 2 applications, but a single application of the Case 1 action eliminates the double-red problem with a trinode restructuring. Therefore, at most one restructuring operation is needed for a red-black tree insertion. ■

Proposition 11.11: *The algorithm for deleting an item from a red-black tree with n items takes $O(\log n)$ time and performs $O(\log n)$ recolorings and at most two restructuring operations.*

Justification: A deletion begins with the standard binary search tree deletion algorithm, which requires time proportional to the height of the tree; for red-black trees, that height is $O(\log n)$. The subsequent rebalancing takes place along an upward path from the parent of a deleted node.

We considered three cases to remedy a resulting black deficit. Case 1 requires a trinode restructuring operation, yet completes the process, so this case is applied at most once. Case 2 may be applied logarithmically many times, but it only involves a recoloring of up to two nodes per application. Case 3 requires a rotation, but this case can only apply once, because if the rotation does not resolve the problem, the very next action will be a terminal application of either Case 1 or Case 2.

In the worst case, there will be $O(\log n)$ recolorings from Case 2, a single rotation from Case 3, and a trinode restructuring from Case 1. ■

11.6.2 Python Implementation

A complete implementation of a RedBlackTreeMap class is provided in Code Fragments 11.15 through 11.17. It inherits from the standard TreeMap class and relies on the balancing framework described in Section 11.2.1.

We begin, in Code Fragment 11.15, by overriding the definition of the nested _Node class to introduce an additional Boolean field to denote the current color of a node. Our constructor intentionally sets the color of a new node to red to be consistent with our approach for inserting items. We define several additional utility functions, at the top of Code Fragment 11.16, that aid in setting the color of nodes and querying various conditions.

When an element has been inserted as a leaf in the tree, the _rebalance_insert hook is called, allowing us the opportunity to modify the tree. The new node is red by default, so we need only look for the special case of the new node being the root (in which case it should be colored black), or the possibility that we have a double-red violation because the new node's parent is also red. To remedy such violations, we closely follow the case analysis described in Section 11.6.1.

The rebalancing after a deletion also follows the case analysis described in Section 11.6.1. An additional challenge is that by the time the _rebalance_hook is called, the old node has already been removed from the tree. That hook is invoked on the *parent* of the removed node. Some of the case analysis depends on knowing about the properties of the removed node. Fortunately, we can reverse engineer that information by relying on the red-black tree properties. In particular, if p denotes the parent of the removed node, it must be that:

- If p has no children, the removed node was a red leaf. (Exercise R-11.26.)
- If p has one child, the removed node was a black leaf, causing a deficit, unless that one remaining child is a red leaf. (Exercise R-11.27.)
- If p has two children, the removed node was a black node with one red child, which was promoted. (Exercise R-11.28.)

```
1  class RedBlackTreeMap(TreeMap):
2    """Sorted map implementation using a red-black tree."""
3    class _Node(TreeMap._Node):
4      """Node class for red-black tree maintains bit that denotes color."""
5      __slots__ = '_red'      # add additional data member to the Node class
6
7      def __init__(self, element, parent=None, left=None, right=None):
8        super().__init__(element, parent, left, right)
9        self._red = True      # new node red by default
```

Code Fragment 11.15: Beginning of the RedBlackTreeMap class. (Continued in Code Fragment 11.16.)

```
10    #------------------------ positional-based utility methods ------------------------
11    # we consider a nonexistent child to be trivially black
12    def _set_red(self, p): p._node._red = True
13    def _set_black(self, p): p._node._red = False
14    def _set_color(self, p, make_red): p._node._red = make_red
15    def _is_red(self, p): return p is not None and p._node._red
16    def _is_red_leaf(self, p): return self._is_red(p) and self.is_leaf(p)
17
18    def _get_red_child(self, p):
19      """Return a red child of p (or None if no such child)."""
20      for child in (self.left(p), self.right(p)):
21        if self._is_red(child):
22          return child
23      return None
24
25    #------------------------ support for insertions ------------------------
26    def _rebalance_insert(self, p):
27      self._resolve_red(p)                          # new node is always red
28
29    def _resolve_red(self, p):
30      if self.is_root(p):
31        self._set_black(p)                          # make root black
32      else:
33        parent = self.parent(p)
34        if self._is_red(parent):                    # double red problem
35          uncle = self.sibling(parent)
36          if not self._is_red(uncle):               # Case 1: misshapen 4-node
37            middle = self._restructure(p)           # do trinode restructuring
38            self._set_black(middle)                 # and then fix colors
39            self._set_red(self.left(middle))
40            self._set_red(self.right(middle))
41          else:                                     # Case 2: overfull 5-node
42            grand = self.parent(parent)
43            self._set_red(grand)                    # grandparent becomes red
44            self._set_black(self.left(grand))       # its children become black
45            self._set_black(self.right(grand))
46            self._resolve_red(grand)                # recur at red grandparent
```

Code Fragment 11.16: Continuation of the RedBlackTreeMap class. (Continued from Code Fragment 11.15, and concluded in Code Fragment 11.17.)

```
47    #------------------------ support for deletions ------------------------
48    def _rebalance_delete(self, p):
49      if len(self) == 1:
50        self._set_black(self.root( ))      # special case: ensure that root is black
51      elif p is not None:
52        n = self.num_children(p)
53        if n == 1:                         # deficit exists unless child is a red leaf
54          c = next(self.children(p))
55          if not self._is_red_leaf(c):
56            self._fix_deficit(p, c)
57        elif n == 2:                       # removed black node with red child
58          if self._is_red_leaf(self.left(p)):
59            self._set_black(self.left(p))
60          else:
61            self._set_black(self.right(p))
62
63    def _fix_deficit(self, z, y):
64      """Resolve black deficit at z, where y is the root of z's heavier subtree."""
65      if not self._is_red(y): # y is black; will apply Case 1 or 2
66        x = self._get_red_child(y)
67        if x is not None: # Case 1: y is black and has red child x; do "transfer"
68          old_color = self._is_red(z)
69          middle = self._restructure(x)
70          self._set_color(middle, old_color)       # middle gets old color of z
71          self._set_black(self.left(middle))       # children become black
72          self._set_black(self.right(middle))
73        else: # Case 2: y is black, but no red children; recolor as "fusion"
74          self._set_red(y)
75          if self._is_red(z):
76            self._set_black(z)                     # this resolves the problem
77          elif not self.is_root(z):
78            self._fix_deficit(self.parent(z), self.sibling(z)) # recur upward
79      else: # Case 3: y is red; rotate misaligned 3-node and repeat
80        self._rotate(y)
81        self._set_black(y)
82        self._set_red(z)
83        if z == self.right(y):
84          self._fix_deficit(z, self.left(z))
85        else:
86          self._fix_deficit(z, self.right(z))
```

Code Fragment 11.17: Conclusion of the RedBlackTreeMap class. (Continued from Code Fragment 11.16.)

11.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-11.1 If we insert the entries $(1,A)$, $(2,B)$, $(3,C)$, $(4,D)$, and $(5,E)$, in this order, into an initially empty binary search tree, what will it look like?

R-11.2 Insert, into an empty binary search tree, entries with keys 30, 40, 24, 58, 48, 26, 11, 13 (in this order). Draw the tree after each insertion.

R-11.3 How many different binary search trees can store the keys $\{1,2,3\}$?

R-11.4 Dr. Amongus claims that the order in which a fixed set of entries is inserted into a binary search tree does not matter—the same tree results every time. Give a small example that proves he is wrong.

R-11.5 Dr. Amongus claims that the order in which a fixed set of entries is inserted into an AVL tree does not matter—the same AVL tree results every time. Give a small example that proves he is wrong.

R-11.6 Our implementation of the TreeMap._subtree_search utility, from Code Fragment 11.4, relies on recursion. For a large unbalanced tree, Python's default limit on recursive depth may be prohibitive. Give an alternative implementation of that method that does not rely on the use of recursion.

R-11.7 Do the trinode restructurings in Figures 11.12 and 11.14 result in single or double rotations?

R-11.8 Draw the AVL tree resulting from the insertion of an entry with key 52 into the AVL tree of Figure 11.14b.

R-11.9 Draw the AVL tree resulting from the removal of the entry with key 62 from the AVL tree of Figure 11.14b.

R-11.10 Explain why performing a rotation in an n-node binary tree when using the array-based representation of Section 8.3.2 takes $\Omega(n)$ time.

R-11.11 Give a schematic figure, in the style of Figure 11.13, showing the heights of subtrees during a deletion operation in an AVL tree that triggers a trinode restructuring for the case in which the two children of the node denoted as y start with equal heights. What is the net effect of the height of the rebalanced subtree due to the deletion operation?

R-11.12 Repeat the previous problem, considering the case in which y's children start with different heights.

R-11.13 The rules for a deletion in an AVL tree specifically require that when the two subtrees of the node denoted as y have equal height, child x should be chosen to be "aligned" with y (so that x and y are both left children or both right children). To better understand this requirement, repeat Exercise R-11.11 assuming we picked the misaligned choice of x. Why might there be a problem in restoring the AVL property with that choice?

R-11.14 Perform the following sequence of operations in an initially empty splay tree and draw the tree after each set of operations.

a. Insert keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.
b. Search for keys 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, in this order.
c. Delete keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.

R-11.15 What does a splay tree look like if its entries are accessed in increasing order by their keys?

R-11.16 Is the search tree of Figure 11.23(a) a $(2,4)$ tree? Why or why not?

R-11.17 An alternative way of performing a split at a node w in a $(2,4)$ tree is to partition w into w' and w'', with w' being a 2-node and w'' a 3-node. Which of the keys k_1, k_2, k_3, or k_4 do we store at w's parent? Why?

R-11.18 Dr. Amongus claims that a $(2,4)$ tree storing a set of entries will always have the same structure, regardless of the order in which the entries are inserted. Show that he is wrong.

R-11.19 Draw four different red-black trees that correspond to the same $(2,4)$ tree.

R-11.20 Consider the set of keys $K = \{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15\}$.

a. Draw a $(2,4)$ tree storing K as its keys using the fewest number of nodes.
b. Draw a $(2,4)$ tree storing K as its keys using the maximum number of nodes.

R-11.21 Consider the sequence of keys $(5,16,22,45,2,10,18,30,50,12,1)$. Draw the result of inserting entries with these keys (in the given order) into

a. An initially empty $(2,4)$ tree.
b. An initially empty red-black tree.

R-11.22 For the following statements about red-black trees, provide a justification for each true statement and a counterexample for each false one.

a. A subtree of a red-black tree is itself a red-black tree.
b. A node that does not have a sibling is red.
c. There is a unique $(2,4)$ tree associated with a given red-black tree.
d. There is a unique red-black tree associated with a given $(2,4)$ tree.

R-11.23 Explain why you would get the same output in an inorder listing of the entries in a binary search tree, T, independent of whether T is maintained to be an AVL tree, splay tree, or red-black tree.

R-11.24 Consider a tree T storing 100,000 entries. What is the worst-case height of T in the following cases?

a. T is a binary search tree.
b. T is an AVL tree.
c. T is a splay tree.
d. T is a $(2,4)$ tree.
e. T is a red-black tree.

R-11.25 Draw an example of a red-black tree that is not an AVL tree.

R-11.26 Let T be a red-black tree and let p be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if p has zero children, the removed node was a red leaf.

R-11.27 Let T be a red-black tree and let p be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if p has one child, the deletion has caused a black deficit at p, except for the case when the one remaining child is a red leaf.

R-11.28 Let T be a red-black tree and let p be the position of the *parent* of the original node that is deleted by the standard search tree deletion algorithm. Prove that if p has two children, the removed node was black and had one red child.

Creativity

C-11.29 Explain how to use an AVL tree or a red-black tree to sort n comparable elements in $O(n \log n)$ time in the worst case.

C-11.30 Can we use a splay tree to sort n comparable elements in $O(n \log n)$ time in the ***worst case***? Why or why not?

C-11.31 Repeat Exercise C-10.28 for the TreeMap class.

C-11.32 Show that any n-node binary tree can be converted to any other n-node binary tree using $O(n)$ rotations.

C-11.33 For a key k that is not found in binary search tree T, prove that both the greatest key less than k and the least key greater than k lie on the path traced by the search for k.

C-11.34 In Section 11.1.2 we claim that the find_range method of a binary search tree executes in $O(s+h)$ time where s is the number of items found within the range and h is the height of the tree. Our implementation, in Code Fragment 11.6 begins by searching for the starting key, and then repeatedly calling the after method until reaching the end of the range. Each call to after is guaranteed to run in $O(h)$ time. This suggests a weaker $O(sh)$ bound for find_range, since it involves $O(s)$ calls to after. Prove that this implementation achieves the stronger $O(s+h)$ bound.

C-11.35 Describe how to perform an operation remove_range(start, stop) that removes all the items whose keys fall within range(start, stop) in a sorted map that is implemented with a binary search tree T, and show that this method runs in time $O(s+h)$, where s is the number of items removed and h is the height of T.

C-11.36 Repeat the previous problem using an AVL tree, achieving a running time of $O(s \log n)$. Why doesn't the solution to the previous problem trivially result in an $O(s+\log n)$ algorithm for AVL trees?

C-11.37 Suppose we wish to support a new method count_range(start, stop) that determines how many keys of a sorted map fall in the specified range. We could clearly implement this in $O(s+h)$ time by adapting our approach to find_range. Describe how to modify the search tree structure to support $O(h)$ worst-case time for count_range.

C-11.38 If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to maintain support for the new method?

C-11.39 Draw a schematic of an AVL tree such that a single remove operation could require $\Omega(\log n)$ trinode restructurings (or rotations) from a leaf to the root in order to restore the height-balance property.

C-11.40 In our AVL implementation, each node stores the height of its subtree, which is an arbitrarily large integer. The space usage for an AVL tree can be reduced by instead storing the ***balance factor*** of a node, which is defined as the height of its left subtree minus the height of its right subtree. Thus, the balance factor of a node is always equal to -1, 0, or 1, except during an insertion or removal, when it may become ***temporarily*** equal to -2 or $+2$. Reimplement the AVLTreeMap class storing balance factors rather than subtree heights.

C-11.41 If we maintain a reference to the position of the leftmost node of a binary search tree, then operation find_min can be performed in $O(1)$ time. Describe how the implementation of the other map methods need to be modified to maintain a reference to the leftmost position.

C-11.42 If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to accurately maintain the reference to the leftmost position?

C-11.43 Describe a modification to the binary search tree implementation having worst-case $O(1)$-time performance for methods after(p) and before(p) without adversely affecting the asymptotics of any other methods.

C-11.44 If the approach described in the previous problem were implemented as part of the TreeMap class, what additional modifications (if any) would be necessary to a subclass such as AVLTreeMap in order to maintain the efficiency?

C-11.45 For a standard binary search tree, Table 11.1 claims $O(h)$-time performance for the delete(p) method. Explain why delete(p) would run in $O(1)$ time if given a solution to Exercise C-11.43.

C-11.46 Describe a modification to the binary search tree data structure that would support the following two index-based operations for a sorted map in $O(h)$ time, where h is the height of the tree.

at_index(i): Return the position p of the item at index i of a sorted map.

index_of(p): Return the index i of the item at position p of a sorted map.

C-11.47 Draw a splay tree, T_1, together with the sequence of updates that produced it, and a red-black tree, T_2, on the same set of ten entries, such that a preorder traversal of T_1 would be the same as a preorder traversal of T_2.

C-11.48 Show that the nodes that become temporarily unbalanced in an AVL tree during an insertion may be nonconsecutive on the path from the newly inserted node to the root.

C-11.49 Show that at most one node in an AVL tree becomes temporarily unbalanced after the immediate deletion of a node as part of the standard __delitem__ map operation.

C-11.50 Let T and U be $(2,4)$ trees storing n and m entries, respectively, such that all the entries in T have keys less than the keys of all the entries in U. Describe an $O(\log n + \log m)$-time method for ***joining*** T and U into a single tree that stores all the entries in T and U.

C-11.51 Repeat the previous problem for red-black trees T and U.

C-11.52 Justify Proposition 11.7.

C-11.53 The Boolean indicator used to mark nodes in a red-black tree as being "red" or "black" is not strictly needed when we have distinct keys. Describe a scheme for implementing a red-black tree without adding any extra space to standard binary search tree nodes.

C-11.54 Let T be a red-black tree storing n entries, and let k be the key of an entry in T. Show how to construct from T, in $O(\log n)$ time, two red-black trees T' and T'', such that T' contains all the keys of T less than k, and T'' contains all the keys of T greater than k. This operation destroys T.

C-11.55 Show that the nodes of any AVL tree T can be colored "red" and "black" so that T becomes a red-black tree.

C-11.56 The standard splaying step requires two passes, one downward pass to find the node x to splay, followed by an upward pass to splay the node x. Describe a method for splaying and searching for x in one downward pass. Each substep now requires that you consider the next two nodes in the path down to x, with a possible zig substep performed at the end. Describe how to perform the zig-zig, zig-zag, and zig steps.

C-11.57 Consider a variation of splay trees, called ***half-splay trees***, where splaying a node at depth d stops as soon as the node reaches depth $\lfloor d/2 \rfloor$. Perform an amortized analysis of half-splay trees.

C-11.58 Describe a sequence of accesses to an n-node splay tree T, where n is odd, that results in T consisting of a single chain of nodes such that the path down T alternates between left children and right children.

C-11.59 As a positional structure, our TreeMap implementation has a subtle flaw. A position instance p associated with an key-value pair (k,v) should remain valid as long as that item remains in the map. In particular, that position should be unaffected by calls to insert or delete other items in the collection. Our algorithm for deleting an item from a binary search tree may fail to provide such a guarantee, in particular because of our rule for using the inorder predecessor of a key as a replacement when deleting a key that is located in a node with two children. Given an explicit series of Python commands that demonstrates such a flaw.

C-11.60 How might the TreeMap implementation be changed to avoid the flaw described in the previous problem?

Projects

P-11.61 Perform an experimental study to compare the speed of our AVL tree, splay tree, and red-black tree implementations for various sequences of operations.

P-11.62 Redo the previous exercise, including an implementation of skip lists. (See Exercise P-10.53.)

P-11.63 Implement the Map ADT using a $(2,4)$ tree. (See Section 10.1.1.)

P-11.64 Redo the previous exercise, including all methods of the Sorted Map ADT. (See Section 10.3.)

P-11.65 Redo Exercise P-11.63 providing *positional* support, as we did for binary search trees (Section 11.1.1), so as to include methods first( ), last( ), before(p), after(p), and find_position(k). Each item should have a distinct position in this abstraction, even though several items may be stored at a single node of a tree.

P-11.66 Write a Python class that can take any red-black tree and convert it into its corresponding $(2,4)$ tree and can take any $(2,4)$ tree and convert it into its corresponding red-black tree.

P-11.67 In describing multisets and multimaps in Section 10.5.3, we describe a general approach for adapting a traditional map by storing all duplicates within a secondary container as a value in the map. Give an alternative implementation of a multimap using a binary search tree such that each entry of the map is stored at a distinct node of the tree. With the existence of duplicates, we redefine the search tree property so that all items in the left subtree of a position p with key k have keys that are less than *or equal to* k, while all items in the right subtree of p have keys that are greater than *or equal to* k. Use the public interface given in Code Fragment 10.17.

P-11.68 Prepare an implementation of splay trees that uses top-down splaying as described in Exercise C-11.56. Perform extensive experimental studies to compare its performance to the standard bottom-up splaying implemented in this chapter.

P-11.69 The ***mergeable heap*** ADT is an extension of the priority queue ADT consisting of operations add(k, v), min( ), remove_min( ) and merge(h), where the merge(h) operations performs a union of the mergeable heap h with the present one, incorporating all items into the current one while emptying h. Describe a concrete implementation of the mergeable heap ADT that achieves $O(\log n)$ performance for all its operations, where n denotes the size of the resulting heap for the merge operation.

P-11.70 Write a program that performs a simple n-body simulation, called "Jumping Leprechauns." This simulation involves n leprechauns, numbered 1 to n. It maintains a gold value g_i for each leprechaun i, which begins with each leprechaun starting out with a million dollars worth of gold, that is, $g_i = 1\,000\,000$ for each $i = 1,2,\ldots,n$. In addition, the simulation also maintains, for each leprechaun, i, a place on the horizon, which is represented as a double-precision floating-point number, x_i. In each iteration of the simulation, the simulation processes the leprechauns in order. Processing a leprechaun i during this iteration begins by computing a new place on the horizon for i, which is determined by the assignment

$$x_i = x_i + rg_i,$$

where r is a random floating-point number between -1 and 1. The leprechaun i then steals half the gold from the nearest leprechauns on either side of him and adds this gold to his gold value, g_i. Write a program that can perform a series of iterations in this simulation for a given number, n, of leprechauns. You must maintain the set of horizon positions using a sorted map data structure described in this chapter.

Chapter Notes

Some of the data structures discussed in this chapter are extensively covered by Knuth in his *Sorting and Searching* book [65], and by Mehlhorn in [76]. AVL trees are due to Adel'son-Vel'skii and Landis [2], who invented this class of balanced search trees in 1962. Binary search trees, AVL trees, and hashing are described in Knuth's *Sorting and Searching* [65] book. Average-height analyses for binary search trees can be found in the books by Aho, Hopcroft, and Ullman [6] and Cormen, Leiserson, Rivest and Stein [29]. The handbook by Gonnet and Baeza-Yates [44] contains a number of theoretical and experimental comparisons among map implementations. Aho, Hopcroft, and Ullman [5] discuss $(2,3)$ trees, which are similar to $(2,4)$ trees. Red-black trees were defined by Bayer [10]. Variations and interesting properties of red-black trees are presented in a paper by Guibas and Sedgewick [48]. The reader interested in learning more about different balanced tree data structures is referred to the books by Mehlhorn [76] and Tarjan [95], and the book chapter by Mehlhorn and Tsakalidis [78]. Knuth [65] is excellent additional reading that includes early approaches to balancing trees. Splay trees were invented by Sleator and Tarjan [89] (see also [95]).